BACSE202 · Database Systems · Fall Semester 2026

PolyVote

Evaluation dossier: measured evidence for the integrity, privacy, authentication and performance claims

Everything below was produced by executable scripts in the repository and can be re-run. Where a result comes from documentation rather than from a measurement, it is labelled as such.

28/28	0/399	500/500	21
in-scope attacks detected by the final design	false alarms on clean elections	tallies agreeing with an independent count	figures, plus 18 tables and 20 screenshots

Team: Tharakesharasu Thennarasu (25BCE2699), M Pavan Durga Karthik (25BCE2712), Shivam Malge (25BCE0386) · Guide: Dr. Katari Balakrishna · 4 November 2026
Live demo: https://polyvote-bacse202.onrender.com · Source: https://github.com/Tharakeshtj/polyvote-election-platform

1. Summary

We attacked PolyVote with **31 scenarios** that a rogue database administrator, a ledger-store administrator, or both could carry out, recorded **which individual detector fired** for each one, and derived the behaviour of other design choices from that single record. 28 scenarios are inside the stated attacker model (someone with write access to the stores who does not hold the ledger key); three are outside it and are reported anyway.

100%	**11/31**	**11/31**	**8/31**
in-scope detection, final design (28/28); all 31: 28/31	detected by our own Review 2 design (plain chain + receipt sweep)	detected by a typical CRUD app whose officer compares the counters	prevented by append-only triggers against a writer who cannot drop them

Headline findings

- **The combination matters.** No single defence is enough: the strongest single detector finds 10 of 31 on its own, while the full design finds 28. Removing any one detector loses between 0 and 7 attacks (Figure 3).
- **The secret key is what stops forged and rebuilt chains.** With a plain SHA-256 chain the final design would miss C6, C10, C17, C27; with the HMAC key it catches them (Figure 5).
- **The voter-roster seal closes the ghost-voter gap.** Before it, the design caught 23 of 31 and missed every roster attack (C13 to C17); with it, all of them are caught.
- **Accuracy:** across 399 sweeps of randomised clean elections there were 0 false alarms; when 1, 5 or 20 ballots per category were tampered, the sweep named exactly those ballots (precision and recall 1.0).
- **Correctness:** the instant-runoff tally agreed with an independent implementation that explores every tie-break in 500 of 500 random elections.
- **Passwords:** stored hashes are now salted and slowed (PBKDF2-SHA256, 600,000 rounds, about 361,996 times slower per guess than the unsalted SHA-256 they replace).
- **Performance:** at 100,000 ballots the sweep takes 5,985 ms and the chain check 2,731 ms; the roster seal adds 1629 ms. Casting one ballot costs 251.14 ms at 10,000 ballots (median) with the file ledger.
- **Concurrency:** up to 16 simultaneous voters, 0 errors, and the chain and sweep were intact after every run.

> **How to read the comparison.** PolyVote is not claimed to be stronger than the cryptographic end-to-end systems (Helios, Belenios, ElectionGuard) at proving what a ballot says. The claim is narrower and measured: among the designs we could test, only the combination used here detected every in-scope storage-level attack, and it does so at the cost, size and set-up effort of a small-election tool. Section 4 states where related systems are stronger.

2. Method

2.1 System under test and attacker model

PolyVote stores ballots and voters in an election database (SQLite), receipts in a hash-chained ledger (MongoDB Atlas, or a file here), and voter emails and one-time codes in a third store. The attacker model is the one in the report: a person with **write access to one or both stores** (a rogue administrator) who **does not hold the ledger key** and cannot change the application. Three scenarios step outside it on purpose: two where the attacker holds the key (C28, C29), and one that edits display text that is not sealed (C23). They are included so the limits are on the record.

2.2 Detectors

- **Chain check**: recompute every block's hash and link across the ledger.
- **Sweep rule 1 (modified)**, **2 (deleted)**, **3 (injected)**: compare each ballot in the database with its receipt.
- **Rule 4 (candidate seal)**: candidate names against the roster hash sealed in every block. **Rule 5 (count mismatch)**: voter flags, ballots and receipts must agree.
- **Rule 6 (voter seal, new)**: the voters now in the database against a keyed seal the application wrote as it built the roster, kept in the ledger store.
- **Counters**: an officer comparing the three counters by eye (not automatic).

2.3 One record, many designs

Every scenario was run once with the keyed (HMAC) chain and once with a plain SHA-256 chain, and for each run we stored *which detectors fired* (Table 2). A **design profile** is a subset of detectors plus a key mode, so its detection is read from the record rather than re-implemented. This makes ablations and design comparisons exactly like-for-like. To check the idea, we emulated our own Review 2 design (plain chain, receipt sweep) this way and compared it with the **real Review 2 code** run earlier on scenarios C1 to C13: the two agreed on **13 of 13** scenarios (Table 7).

Table 1. Design profiles and what they detect

Profile	Design	Detectors used	Key	Detected (all 31)	Rate	Detected (in scope)	Rate (in scope)	Missed
P0	CRUD app: officer compares the counters	Eyeballing counters	unkeyed	11/31	35.5%	11/28	39.3%	C1 C2 C6 C7 C10 C11 C13 C14 C15 C16 C17 C20 C21 C22 C23 C24 C25 C27 C28 C29
P1	CRUD app + automatic count check	Rule 5 count mismatch	unkeyed	10/31	32.3%	10/28	35.7%	C1 C2 C6 C7 C10 C11 C13 C14 C15 C16 C17 C19 C20 C21 C22 C23 C24 C25 C27 C28 C29
P2	Plain hash chain only	Chain check	unkeyed	6/31	19.4%	6/28	21.4%	C1 C2 C3 C4 C5 C6 C9 C10 C11 C12 C13 C14 C15 C16 C17 C18 C23 C24 C25 C26 C27 C28 C29 C30 C31
P3	Plain chain + receipt sweep (Review 2 design)	Chain check + Rule 1 modified + Rule 2 deleted	unkeyed	11/31	35.5%	11/28	39.3%	C5 C6 C9 C10 C11 C12 C13 C14 C15 C16 C17 C18 C23 C24 C25 C26 C27 C28 C29 C30
P4	Plain chain + full sweep	Chain check + Rule 1 modified + Rule 2 deleted + Rule 3 injected + Rule 4 candidate seal + Rule 5 count mismatch + Rule 6 voter seal	unkeyed	24/31	77.4%	24/28	85.7%	C6 C10 C17 C23 C27 C28 C29
P5	Keyed chain only	Chain check	keyed	9/31	29.0%	9/28	32.1%	C1 C2 C3 C4 C5 C9 C11 C12 C13 C14 C15 C16 C17 C18 C23 C24 C25 C26 C28 C29 C30 C31
P6	Keyed chain + sweep rules 1-5 (previous final design)	Chain check + Rule 1 modified + Rule 2 deleted + Rule 3 injected + Rule 4 candidate seal + Rule 5 count mismatch	keyed	23/31	74.2%	23/28	82.1%	C13 C14 C15 C16 C17 C23 C28 C29
P7	PolyVote final (keyed chain + 6 sweep rules)	Chain check + Rule 1 modified + Rule 2 deleted + Rule 3 injected + Rule 4 candidate seal + Rule 5 count mismatch + Rule 6 voter seal	keyed	28/31	90.3%	28/28	100.0%	C23 C28 C29

2.4 Environment and reproducibility

Machine	Windows-11-10.0.26300-SP0
Python	3.12.10
Run date	2026-11-04 00:12:04
Total run time	569 s
Isolation	a throwaway copy of the application, its own database and ledger files; Atlas and the real data are never touched
Commands	`python experiments/full_eval/run_all.py`, `static_metrics.py`, `shots.py`, `figures.py`, `dossier.py`
Raw data	`experiments/results/full/*.json, *.csv`

3. Results

3.1 Attack detection

The final design detected **28 of 31** scenarios (28 of the 28 in scope). The three it did not detect are the two insider-with-key attacks and the unsealed platform text.



Figure 1. Detection of 31 attacks by eight design profiles. Rows are attack scenarios; columns are design profiles P0 to P7 (defined in Table 3). Green cells are detected. The count above each column is the number of the 31 scenarios it detects.

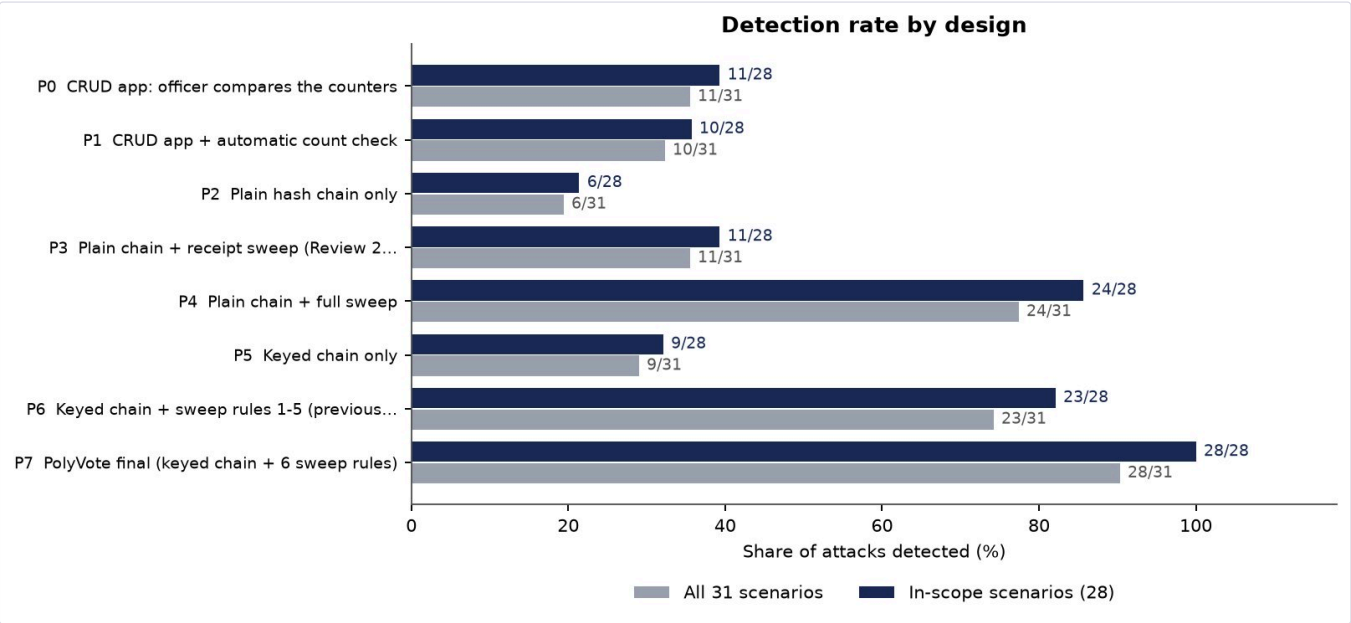


Figure 2. Detection rate by design. Share of attack scenarios each design profile detects. In-scope excludes the three scenarios outside the stated attacker model (Table 1).

Table 2. The 31 attack scenarios

ID	Attack	Attacker capability	In the attacker model	Note
C1	Swap the 1st and 2nd choice on one ballot	Election database	Yes	
C2	Replace the first-choice candidate on one ballot	Election database	Yes	
C3	Delete one ballot	Election database	Yes	
C4	Delete a ballot and clear a voted flag to hide the gap	Election database	Yes	
C5	Inject a ballot with no receipt	Election database	Yes	
C6	Ballot stuffing: inject ballot, forge a plain-hash receipt, set a voted flag	Both stores, no key	Yes	
C7	Edit a ledger block without repairing the chain	Ledger store	Yes	
C8	Delete a block from the middle of the ledger	Ledger store	Yes	
C9	Truncate the last block of the ledger	Ledger store	Yes	
C10	Change a ballot and rebuild the whole chain (attacker has no key)	Both stores, no key	Yes	
C11	Swap two candidates' names after voting	Election database	Yes	
C12	Reset a voted flag, then let that voter cast a second ballot	Database + public API	Yes	
C13	Add a ghost voter to the roster and vote as them through the API	Database + public API	Yes	
C14	Delete an unvoted voter from the roster (disenfranchise)	Election database	Yes	
C15	Rename an unvoted voter's registration number	Election database	Yes	
C16	Delete the roster seal, then add a ghost voter and vote	Both stores, no key	Yes	
C17	Forge the roster seal without the key, then add a ghost voter and vote	Both stores, no key	Yes	
C18	Wipe the whole ledger	Ledger store	Yes	
C19	Append a copy of an existing ledger block (replay a receipt)	Ledger store	Yes	
C20	Swap two ledger blocks (reorder)	Ledger store	Yes	
C21	Edit the timestamp of a ledger block	Ledger store	Yes	
C22	Change a ballot and its block hash but leave the chain unrepaired	Both stores, no key	Yes	
C23	Edit a candidate's platform statement	Election database	No	Not covered by design: only the candidate id and name are sealed.
C24	Add a candidate after voting	Election database	Yes	
C25	Delete a candidate row after voting (foreign keys off)	Election database	Yes	
C26	Mark an unvoted voter as voted with no ballot (disenfranchise)	Election database	Yes	
C27	Erase a vote completely: delete the ballot, its block and the flag, rebuild the chain	Both stores, no key	Yes	
C28	Insider WITH the ledger key: change a ballot and rebuild the chain with the key	Both stores + the key	No	Outside the attacker model: needs the secret key.
C29	Insider WITH the key: add a ghost voter and ballot, forge the receipt, re-seal the roster	Both stores + the key	No	Outside the attacker model: needs the secret key.
C30	Roll the ledger store back to an older backup	Ledger store	Yes	
C31	Roll the election database back to an older backup	Election database	Yes	

3.2 Which detector catches what

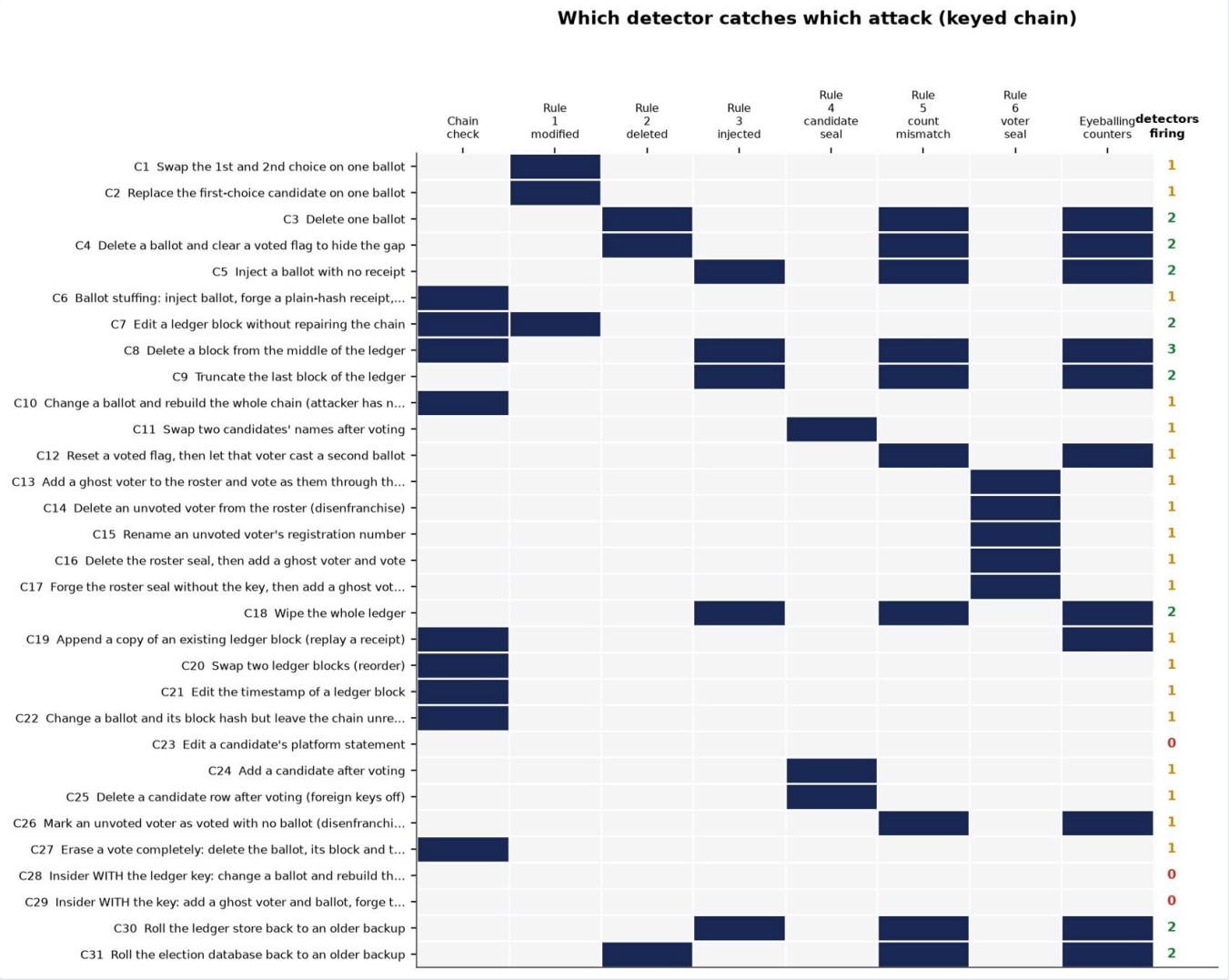


Figure 3. Detector firing matrix. Dark cells show which detectors fire for each attack. The number at the right is how many automatic detectors fire; two or more means the attack is caught redundantly.

Most attacks are caught by more than one detector; the ones caught by only one are the ones to watch if that detector is ever weakened.

3.3 Ablation

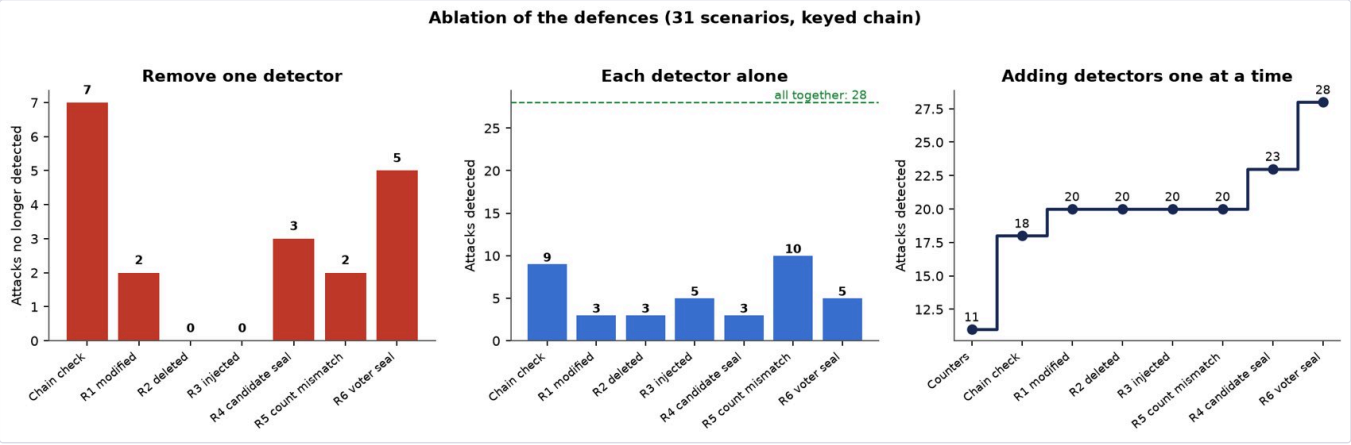


Figure 4. Ablation of the defences. Left: attacks lost when one detector is removed. Middle: attacks each detector finds on its own. Right: cumulative detection as detectors are added in the order shown.

Table 3. Leave-one-out ablation

Removed detector	Still detected	Lost attacks
Chain check	21/31	C6 C10 C19 C20 C21 C22 C27
Rule 1 modified	26/31	C1 C2
Rule 2 deleted	28/31	-
Rule 3 injected	28/31	-
Rule 4 candidate seal	25/31	C11 C24 C25

Removed detector	Still detected	Lost attacks
Rule 5 count mismatch	26/31	C12 C26
Rule 6 voter seal	23/31	C13 C14 C15 C16 C17

Table 4. Each detector on its own

Detector alone	Detected	Attacks
Chain check	9/31	C6 C7 C8 C10 C19 C20 C21 C22 C27
Rule 1 modified	3/31	C1 C2 C7
Rule 2 deleted	3/31	C3 C4 C31
Rule 3 injected	5/31	C5 C8 C9 C18 C30
Rule 4 candidate seal	3/31	C11 C24 C25
Rule 5 count mismatch	10/31	C3 C4 C5 C8 C9 C12 C18 C26 C30 C31
Rule 6 voter seal	5/31	C13 C14 C15 C16 C17

3.4 What the key and the seal add

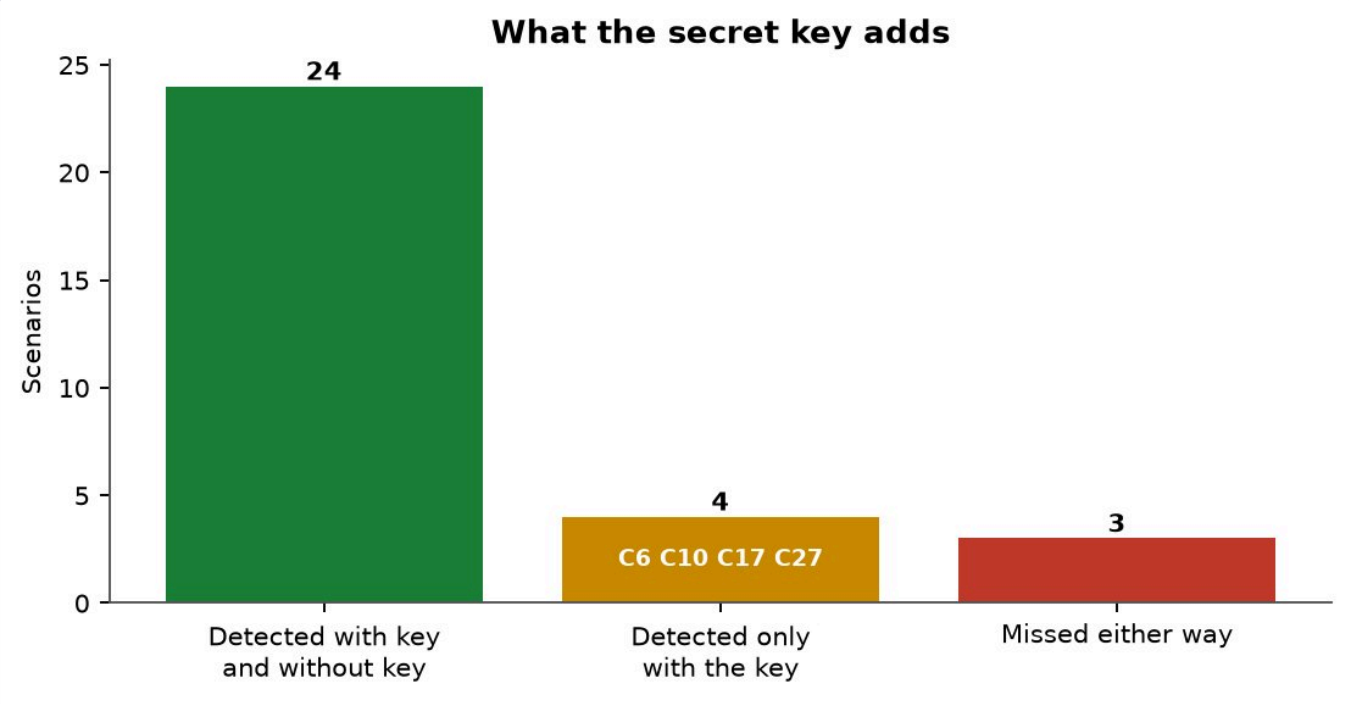


Figure 5. Value of the keyed chain. Scenarios detected by the full design with the HMAC key versus the same design with a plain SHA-256 chain. The ids in the amber bar are attacks that only fail when the attacker does not hold the key.

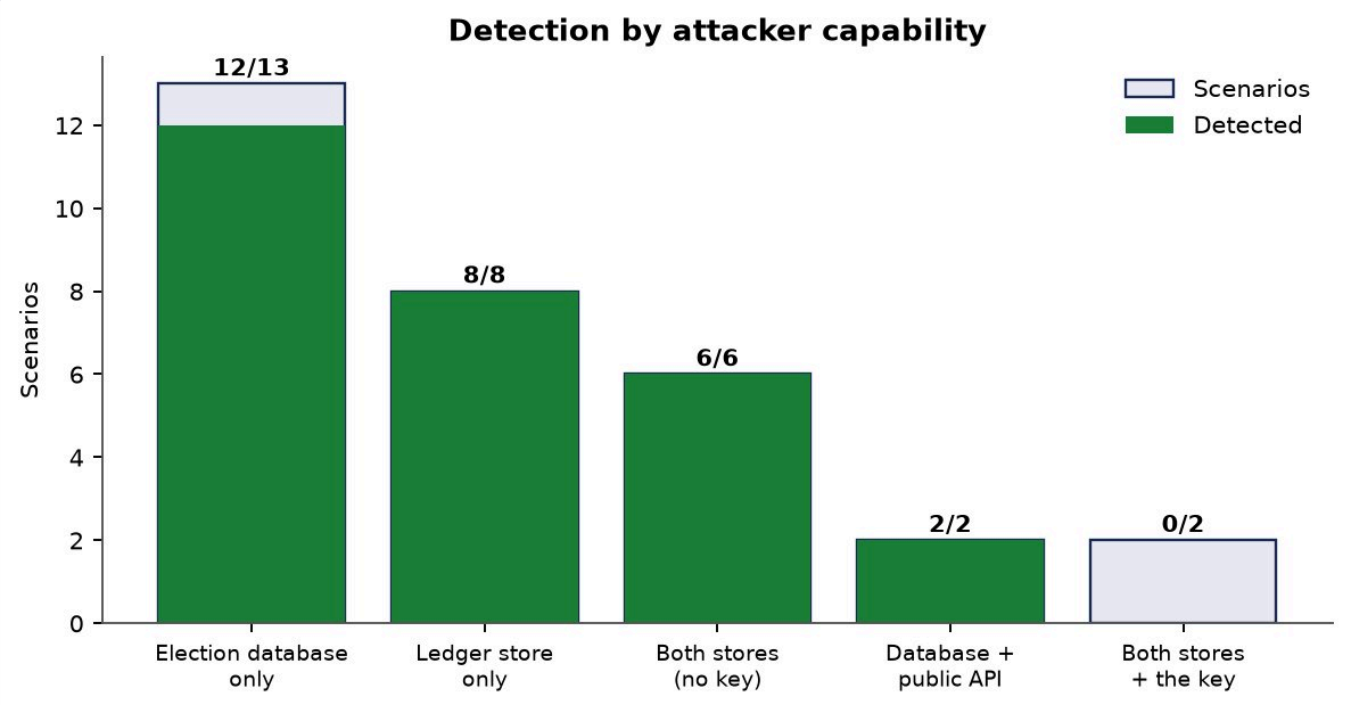


Figure 6. Detection by attacker capability. Every scenario where the attacker lacks the ledger key is detected. The scenarios with the key (and one scenario about unsealed display text) are outside the attacker model and are reported openly.

3.5 Prevention versus detection

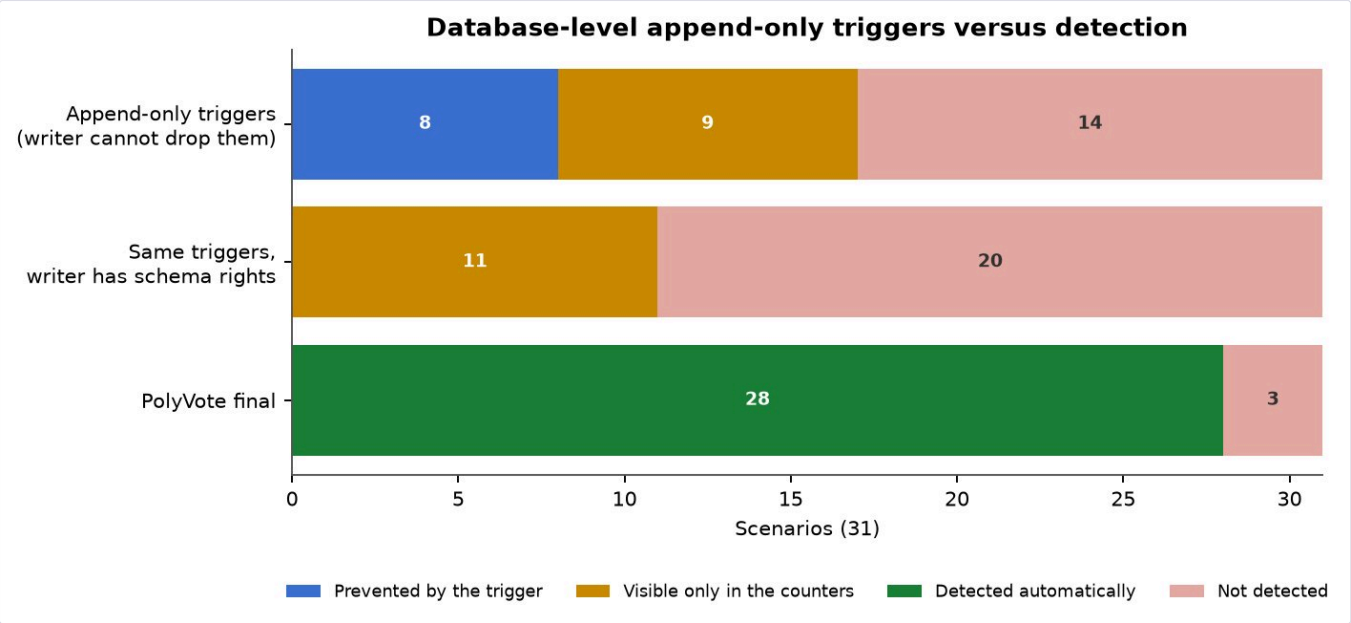


Figure 7. Append-only triggers versus detection. Triggers on the ballot table stop edits and deletions by a writer who cannot drop them, but they do not stop injection, ledger tampering or roster changes, and a writer with schema rights removes them. Detection covers what prevention cannot.

Table 5. Append-only triggers on the ballot table

Class	Count	Scenarios
Prevented by append-only triggers	8	C1 C2 C3 C4 C10 C22 C27 C28
Visible only in the counters	9	C5 C8 C9 C12 C18 C19 C26 C30 C31
Neither	14	C6 C7 C11 C13 C14 C15 C16 C17 C20 C21 C23 C24 C25 C29

Append-only triggers are a useful first line (they stop edits and deletions by a limited writer) but they do not cover injection, the ledger store or the roster, and a writer with schema rights removes them. Detection and prevention are complementary; this design implements both (the database already carries a one-vote trigger and CHECK constraints, Section 3.8).

3.6 Accuracy of the sweep

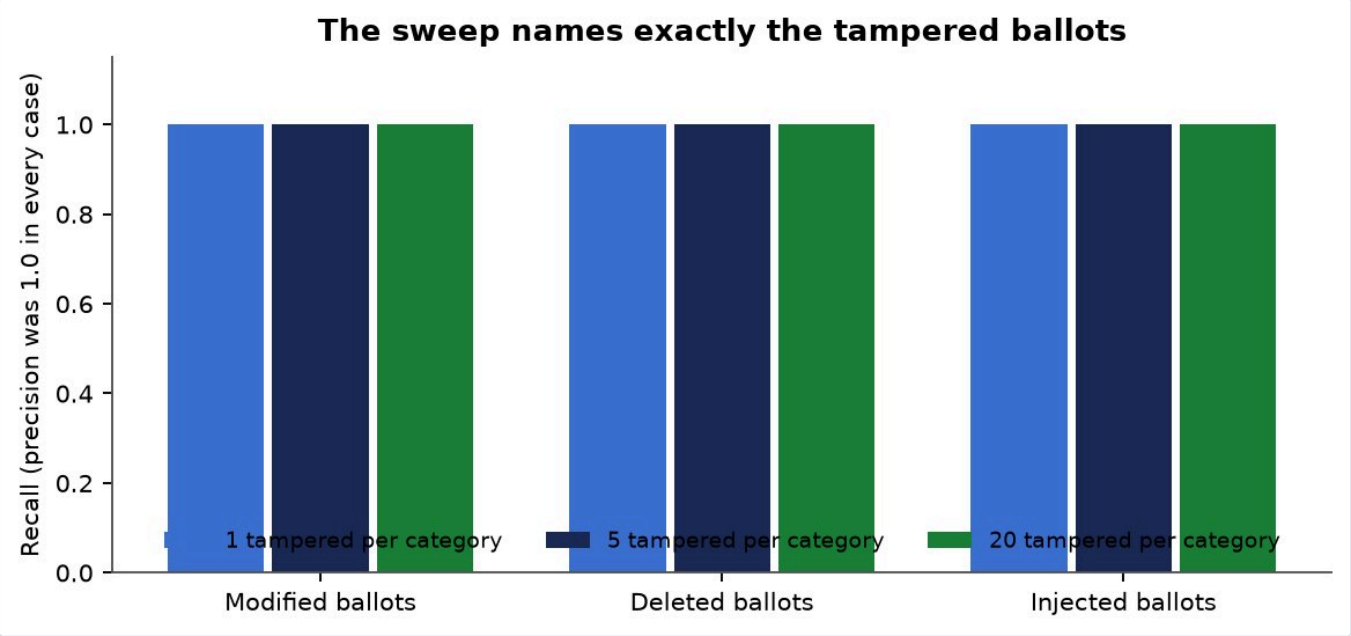


Figure 8. Precision and recall of anomaly identification. With 1, 5 and 20 tampered ballots per category in a 150-ballot election, the sweep reports precisely the tampered tokens: no false positives and no misses.

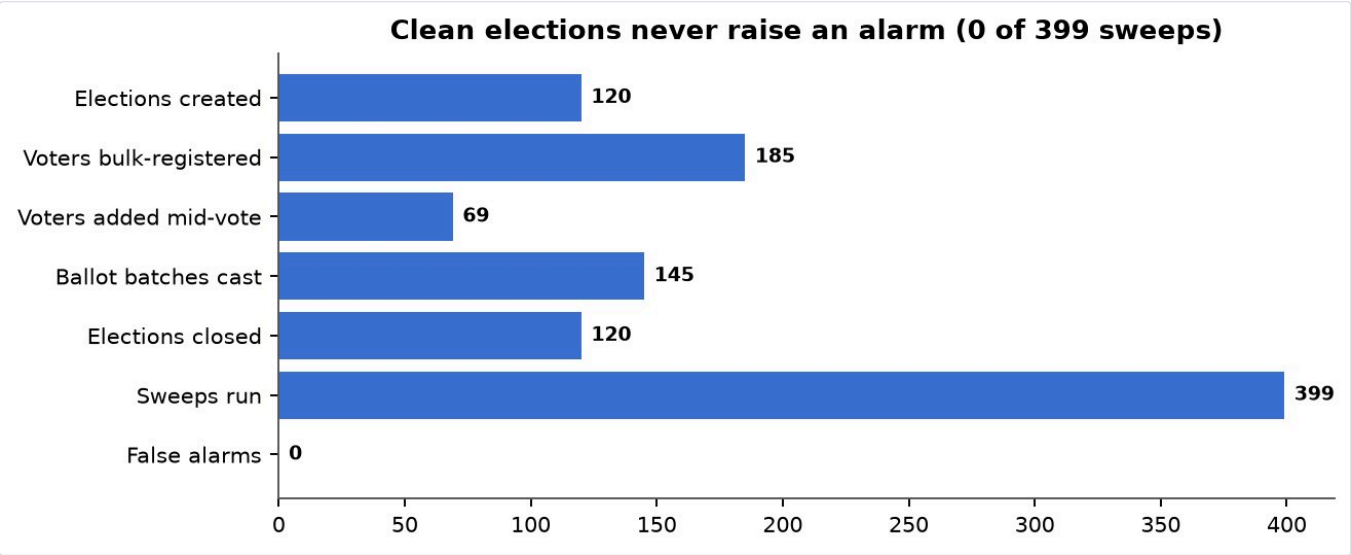


Figure 9. False alarms on clean elections. Randomised legitimate use (bulk registration, late voters, ballots, closing) followed by a sweep and chain check each time, with and without the key.

Table 6. Precision and recall

Tampered per category	Category	True	Reported	True positives	Precision	Recall
1	modified	1	1	1	1.0	1.0
1	deleted	1	1	1	1.0	1.0
1	injected	1	1	1	1.0	1.0
5	modified	5	5	5	1.0	1.0
5	deleted	5	5	5	1.0	1.0
5	injected	5	5	5	1.0	1.0
20	modified	20	20	20	1.0	1.0
20	deleted	20	20	20	1.0	1.0
20	injected	20	20	20	1.0	1.0

Table 7. False alarms on clean elections

Measure	Value
Clean elections	120
Sweeps run	399
False alarms	0
False-positive rate	0.0
Operations: bulk_register	185
Operations: add_voter	69
Operations: add_candidate	120
Operations: simulate	145
Operations: close	120

3.7 Audit time

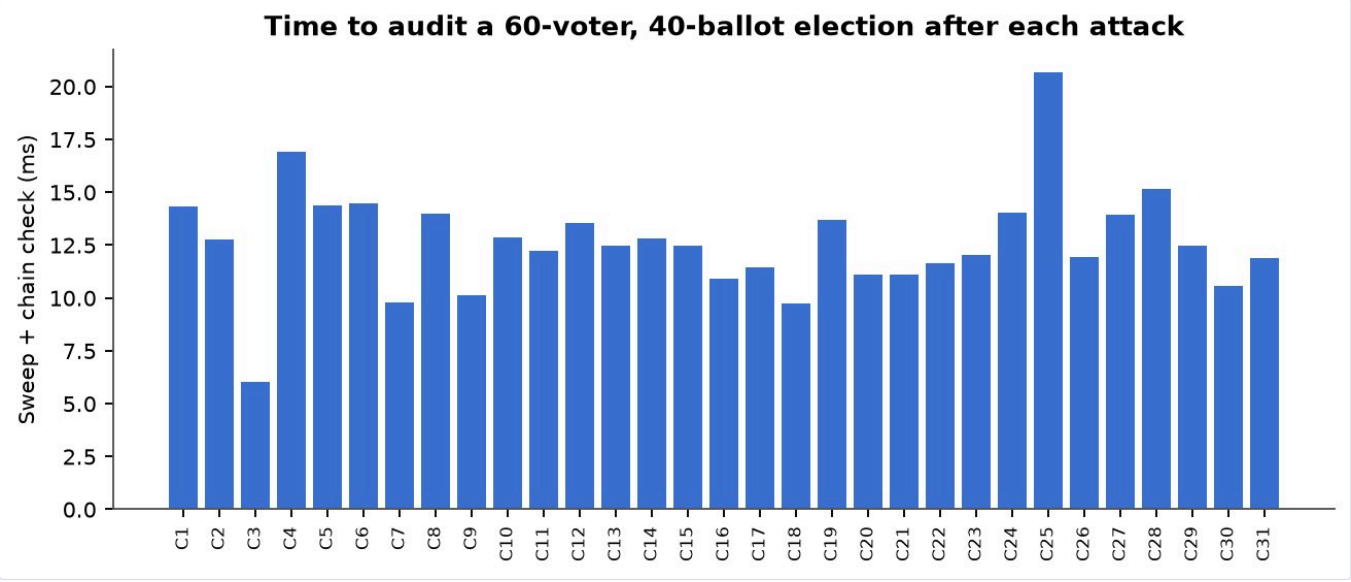


Figure 10. Audit time per scenario. Time for the integrity sweep plus the chain check on the small test election after each attack.

3.8 Database rules, abuse of the API and the one-time code

From the core suite re-run on the final code: database-level prevention 6/7, API abuse refused 13/13, one-time-code tests passed 14/14.

Table. Database-level prevention

ID	Attempt	Mechanism	Result	Database message
A1	Mark an already-voted voter as voted again	trigger	prevented	DBMS Trigger Violation: Voter has already submitted a ballot.
A2	Insert a rank outside 1-3 (rank 4)	CHECK	prevented	CHECK constraint failed: preference_rank BETWEEN 1 AND 3
A3	Rank the same candidate twice on one ballot	composite PK	prevented	UNIQUE constraint failed: BALLOT_RANKS.ballot_token, BALLOT_RANKS.candidate_id
A4	Rank a candidate that does not exist (FK enforcement on)	FK	prevented	FOREIGN KEY constraint failed
A5	Rank a nonexistent candidate from a default connection (FK pragma off)	FK	not_prevented	statement succeeded
A6	Register the same reg. number twice in one election	UNIQUE	prevented	UNIQUE constraint failed: VOTER_PARTICIPATION.student_reg_no, VOTER_PARTICIPATION.election
A7	Set has_voted to an invalid value (2)	CHECK	prevented	CHECK constraint failed: has_voted IN (0, 1)

Table. Abuse through the public API

ID	Attempt	Expected	Result	Detail
B1	Log in with a wrong password	rejected (401)	prevented	HTTP 401
B2	Log in with a reg. number not on the roster	rejected (404)	prevented	HTTP 404
B3	SQL injection in the reg. number field	rejected	prevented	HTTP 404
B4	Replay a session token after the ballot was cast	rejected (401)	prevented	cast 200, replay 401
B5	Log in again after voting	rejected (409)	prevented	HTTP 409
B6	Cast a ballot with no session	rejected (401)	prevented	HTTP 401
B7	Use a voter token on an admin endpoint	rejected (403)	prevented	HTTP 403
B8	Rank another election's candidates	rejected (422)	prevented	HTTP 422
B9	Rank the same candidate twice via the API	rejected (422)	prevented	HTTP 422
B10	Submit an incomplete ranking	rejected (422)	prevented	HTTP 422
B11	20 simultaneous casts from the same voter	exactly one accepted	prevented	1 accepted, 1 ballot rows for the voter
B12	Log in after the election is closed	rejected (409)	prevented	HTTP 409
B13	Re-open a closed election	rejected (409)	prevented	HTTP 409

Table. Emailed one-time code

ID	Test	Expected	Result	Detail
F1	Correct password alone does not create a voting session	no token issued; challenge id is not a session	passed	login 200, challenge used as bearer: 401
F2	Correct emailed code unlocks the ballot	session issued, ballot accepted	passed	verify 200, cast 200
F3	A used code cannot be replayed	rejected (410)	passed	HTTP 410
F4	Three wrong codes lock the challenge	401, 401, 410 then the right code is refused	passed	[401, 401, 410], then correct code: 410
F5	An expired code is refused	rejected (410)	passed	HTTP 410
F6	Resend is rate-limited to one per 30 seconds	immediate resend 429, later resend 200	passed	429 then 200
F7	A resent code invalidates the previous one	old code rejected	passed	old code: HTTP 401
F8	A challenge allows at most 5 codes	sixth send refused (429)	passed	HTTP 429
F9	A voter with no email on file cannot proceed	409 with a clear message	passed	HTTP 409
F10	Plaintext codes are absent from the verification database	code not found in any table	passed	checked OTP_CHALLENGE, MAIL_LOG, VOTER_CONTACT
F11	Repeated sign-ins are rate-limited	sixth challenge in 15 min refused (429)	passed	[200, 200, 200, 200, 200, 429, 429]
F12	Codes are 6 digits including leading zeros	all length 6, mostly distinct	passed	4993 distinct of 5000
F13	A failed email send does not break sign-in	challenge still issued; failure logged; code stays in the outbox	passed	login 200, mail status "failed: ConnectionRefusedError"
F14	Voter email is not stored in the election database	no email column in any election table	passed	election tables: ['ADMIN_USERS', 'BALLOT_RANKS', 'CANDIDATE', 'ELECTION', 'ORGANIZATION', 'VOTER_PAR

Note on one test. The earlier version of test A3 (rank the same candidate twice) picked a candidate at random and so passed only when that candidate happened to be on the chosen ballot. It now uses a candidate that is on the ballot, which is the attack the test describes; the result (refused by the composite primary key) is unchanged, but the earlier pass was partly luck and we say so.

The one gap in this table. A5: SQLite enforces foreign keys only on connections that turn them on. The application does; a default console does not, so a nonexistent candidate can be inserted from a default connection. The sweep reports it as an injected ballot; a server database removes the gap.

3.9 Passwords

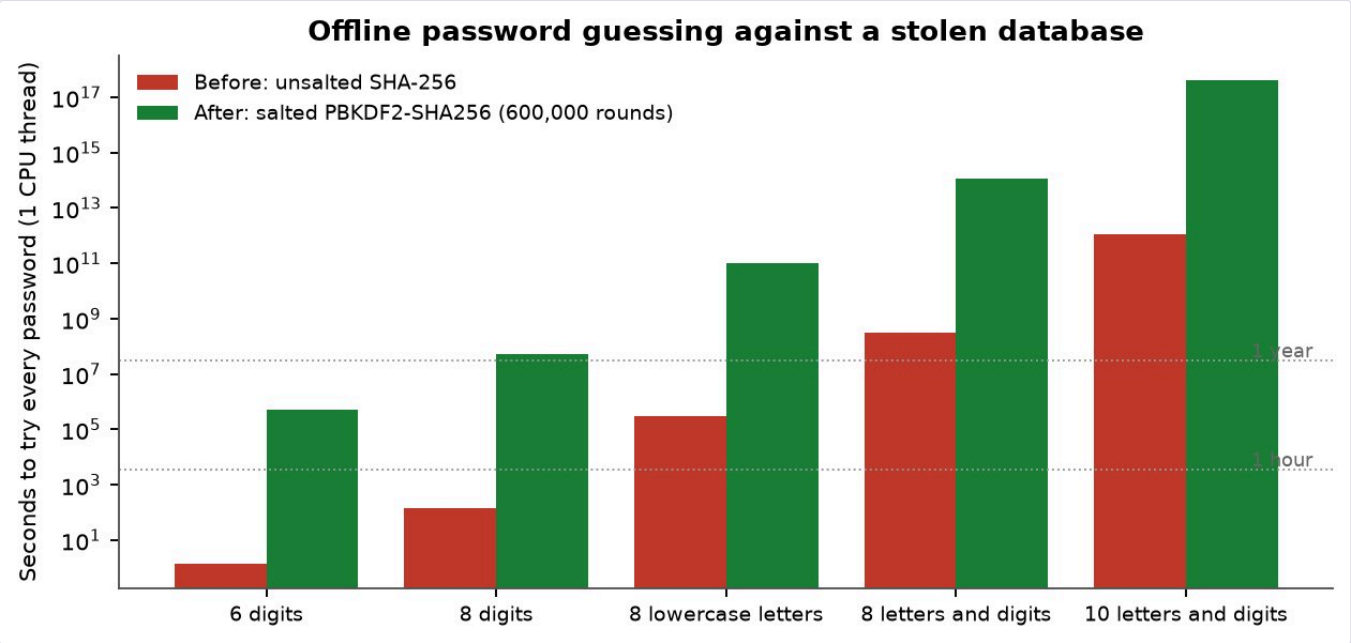


Figure 11. Password hash strength. Time to try every password in a character class on one CPU thread. The new format is about 361,996 times slower per guess and every hash has its own random salt, so tables built once cannot be reused.

Table 8. Offline guessing cost

Password class	Unsalted SHA-256 (before)	Salted PBKDF2 (after)
6 digits	1.4 seconds	5.8 days
8 digits	2.3 minutes	1.6 years
8 lowercase letters	3.3 days	3,303 years
8 letters and digits	9.5 years	3,453,747 years
10 letters and digits	36,675 years	13,276,201,859 years
Hashes per second, 1 CPU thread	725,672	2.0
Verify time at login	<0.01 ms	477.5 ms

Why this changed. In Review 2, passwords were stored as unsalted SHA-256, which an attacker who steals the database can reverse for common passwords almost instantly and with shared tables. They are now stored as `pbkdf2_sha256$rounds$salt$hash` with a random 16-byte salt per password. Existing rows still sign in and are re-stored in the new format on their next successful login. Verification uses a constant-time comparison.

Cost: a password check takes about 477.5 ms on the test laptop (496.25 ms median over complete sign-ins at 600,000 rounds). The rounds can be tuned with `PASSWORD_HASH_ITERATIONS` for slower hosts.

3.10 Voter privacy

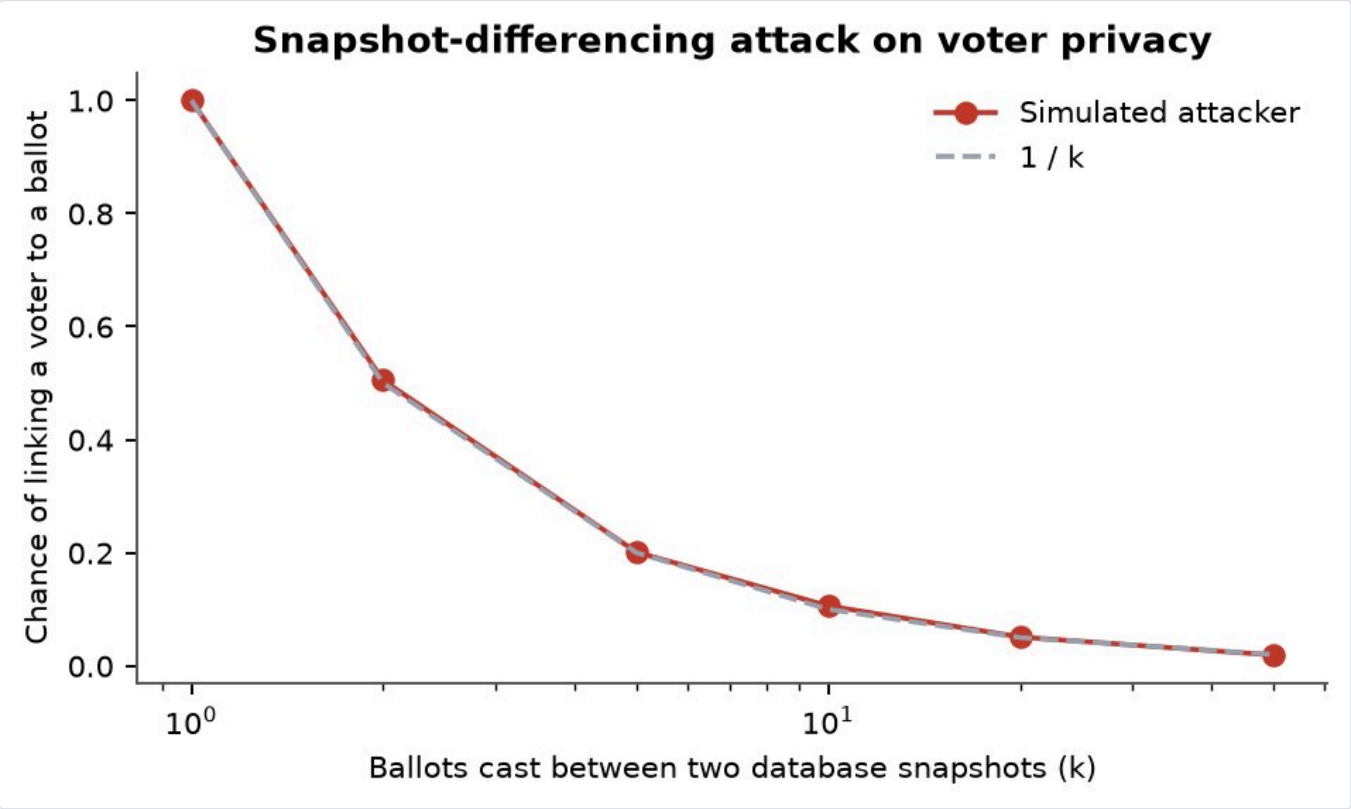


Figure 12. Snapshot differencing. An attacker who copies the database before and after a single cast can link that voter to that ballot; the anonymity set is the number of casts between copies. This is a known limit of any design that stores a voted flag.

Table 9. Voter privacy checks

Check	Result
Columns shared by ballot and voter tables	none
Timestamp on the ballot table	yes (ledger only)
Ledger block fields	ballot_token, curr_hash, election_id, index, prev_hash, roster_hash, timestamp, vote_data_hash, voters_sealed
Ordering attack on 150 API casts in random voter order	0 correct guesses (chance: about 1.0)
Simulation tool casts in roster order	block order mirrors roster order; demo tool only

3.11 Tally correctness

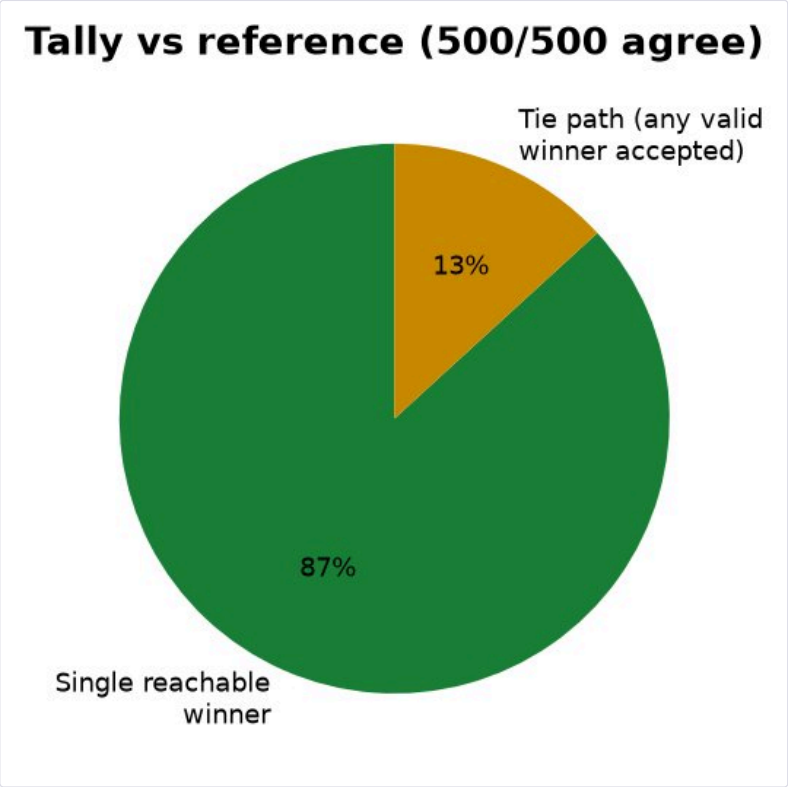


Figure 13. Tally correctness. Random elections (3-8 candidates, 5-300 ballots) counted by the application and by an independent implementation that explores every possible tie-break.

Table 10. Tally correctness

Measure	Value
Random elections	500
Agree with the reference	500
Elections with a tie path	66
Disagreements	0

3.12 Performance and scaling

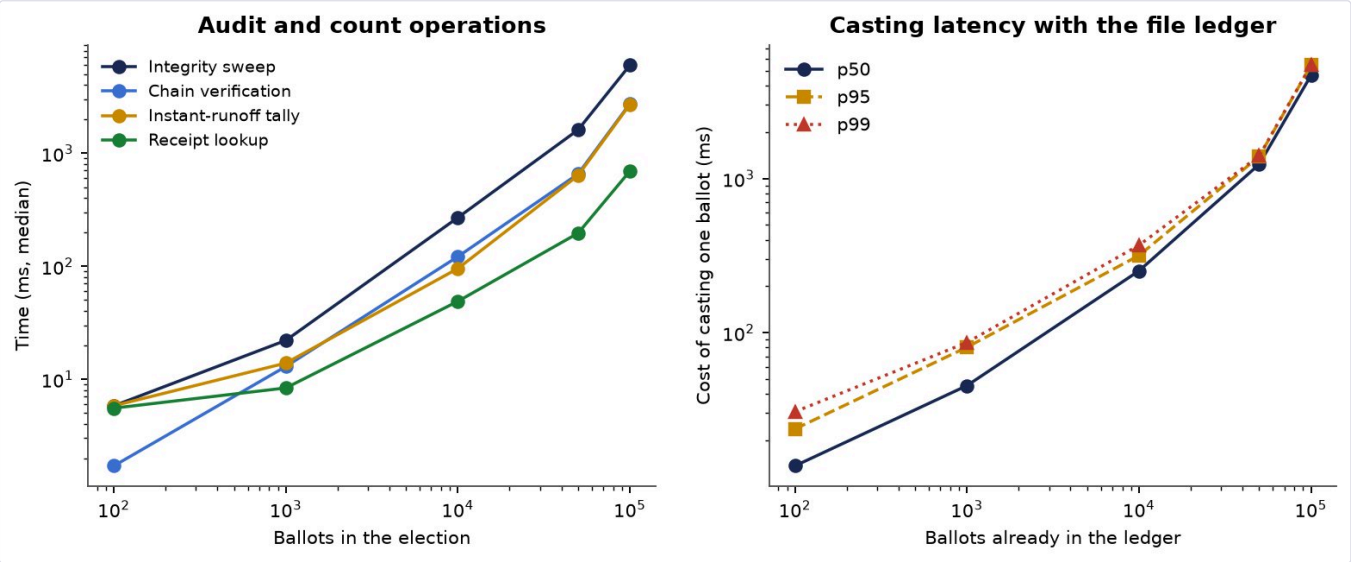


Figure 14. Scaling with election size. Left: median time of the sweep, chain check, tally and receipt lookup from 100 to 100,000 ballots (log scales). Right: latency percentiles of casting one ballot; the file ledger rewrites the file on each append, so cost grows with size.

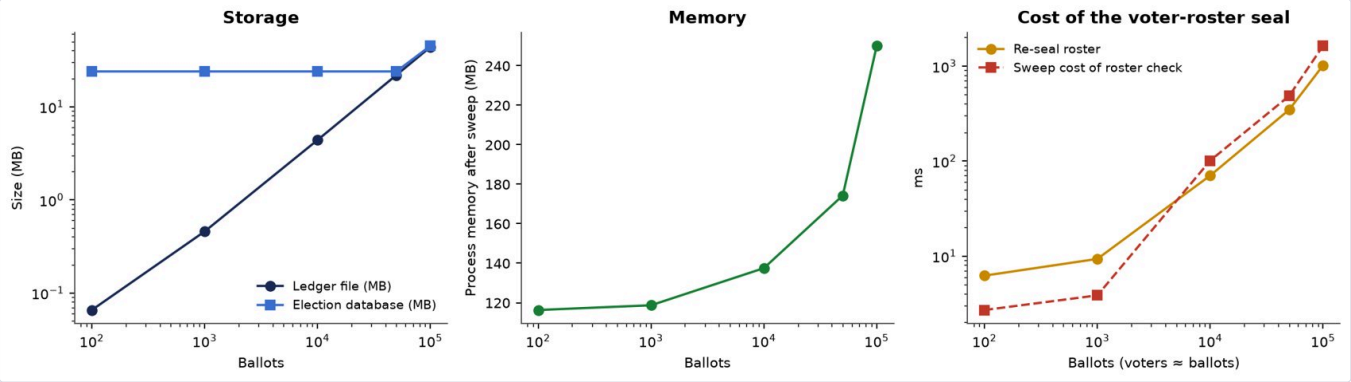


Figure 15. Storage, memory and roster-seal overhead. Storage grows linearly; memory is the process size after a sweep; the roster seal costs milliseconds even at 100,000 voters.

Table 11. Scaling (keyed chain, file ledger)

Ballots	Sweep ms	Sweep without seal ms	Chain check ms	Tally ms	Receipt ms	Cast p50	Cast p95	Cast p99	Ledger KB	DB KB	Memory MB
100	5.81	3.11	1.71	5.82	5.54	13.61	23.63	30.56	67.1	24436.0	116.3
1000	22.02	18.17	12.94	13.87	8.36	45.17	80.19	86.07	467.9	24436.0	118.7
10000	268.59	167.6	121.26	95.22	48.81	251.14	315.87	368.12	4484.5	24436.0	137.5
50000	1604.03	1120.57	652.73	633.65	195.56	1234.7	1385.48	1399.27	22359.3	24436.0	174.3
100000	5984.75	4356.05	2730.54	2688.9	692.14	4658.2	5425.65	5432.47	44722.6	46024.0	250.1

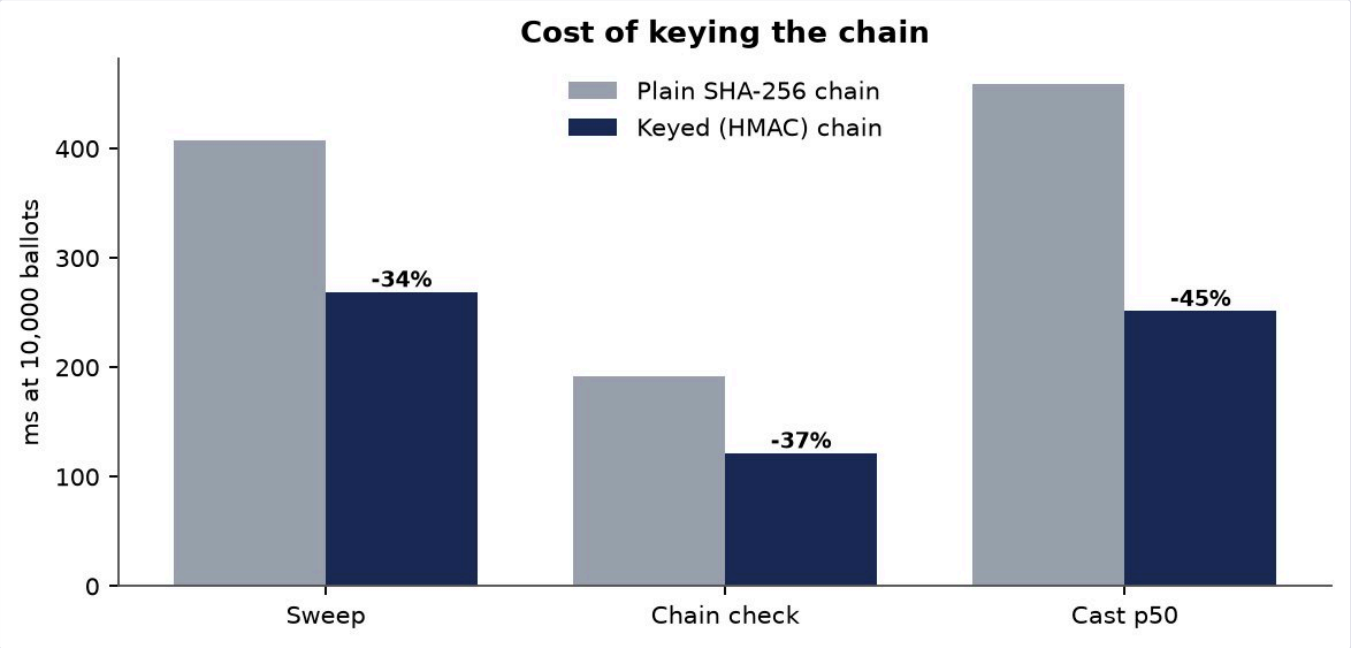


Figure 16. Overhead of the keyed chain. The same operations with a plain SHA-256 chain and with the HMAC-keyed chain at the largest size measured in both modes.

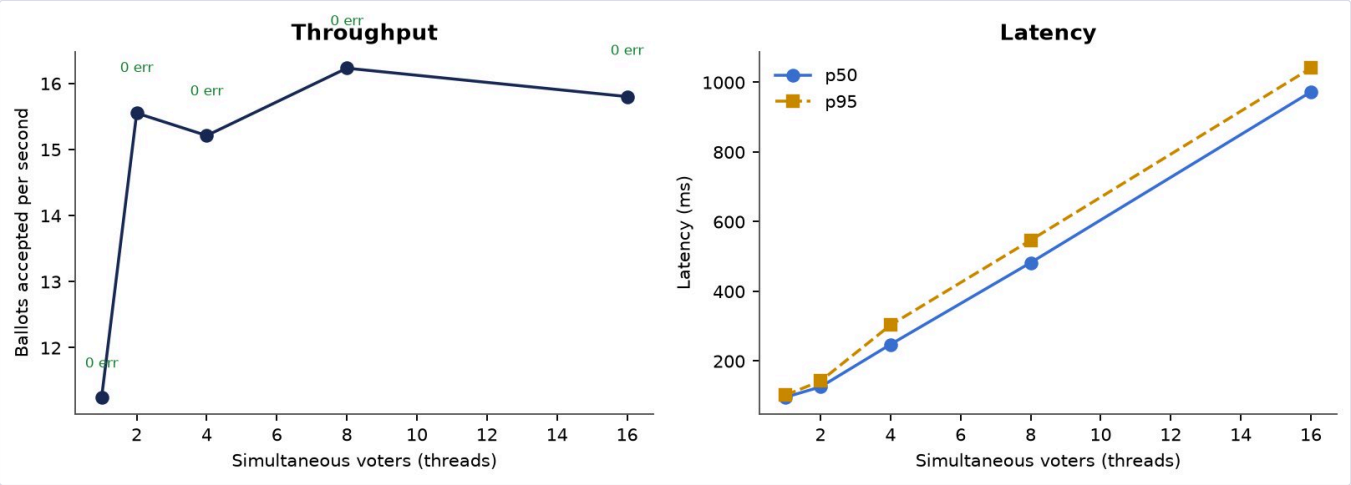


Figure 17. Concurrent voting. A write lock serialises ballot casts, so throughput stays flat while latency grows with the number of simultaneous voters. After every run the chain verified and the sweep passed, with zero errors.

Table 12. Concurrency

Threads	Casts	Wall s	Casts/s	p50 ms	p95 ms	Errors	Chain ok	Sweep ok
1	6	0.53	11.25	95.99	101.9	0	True	True
2	12	0.77	15.55	125.64	142.25	0	True	True
4	24	1.58	15.21	247.57	303.18	0	True	True
8	48	2.96	16.23	482.17	546.71	0	True	True
16	96	6.08	15.8	972.23	1040.39	0	True	True

MongoDB Atlas ledger

100 ballots in the ledger	append median 135.9 ms, 90th percentile 157.4 ms
1000 ballots in the ledger	append median 147.2 ms, 90th percentile 170.4 ms
10000 ballots in the ledger	append median 139.4 ms, 90th percentile 163.5 ms

Measured earlier against a scratch database on Atlas over a mobile hotspot (experiments/bench_mongo.py).

3.13 Security probes

Table 13. Security probes

Probe	Result	Verdict
Brute-force password guesses	40 attempts, 0.9/s, locked out: False	Limitation: no lock-out on the password step (the emailed code is a second factor)
Fuzzing (hostile and malformed input)	117 requests, 0 server errors, tables intact: True	Pass
Admin endpoints without a token	elections: 401, integrity: 401, voters: 401, admins: 401, ledger: 401, outbox: 404	Pass
User enumeration	unknown voter 404, wrong password 401	Limitation: a roster lookup reveals whether a number is registered

Probe	Result	Verdict
Session expiry	session fields: role, username	Limitation: sessions live in memory until sign-out or restart
API security headers	content-security-policy: absent, x-content-type-options: absent, x-frame-options: absent, referrer-policy: absent, permissions-policy: absent, strict-transport-security: absent, cache-control: absent	Limitation (see live-site headers)
CORS from a foreign origin	None	Pass
Receipt token guessing	122 random bits (UUID v4)	Pass

Honest limitations found by the probes. There is no lock-out on repeated wrong passwords (the emailed code is the second factor and the password hash is slow, but a lock-out is future work); sessions do not expire until sign-out or restart; the sign-in distinguishes an unknown registration number from a wrong password; and the API sets no security headers of its own. None of these affects the integrity results above; they are listed so a reader can weigh them.

3.14 Front end, accessibility and live deployment

Table. Automated WCAG 2.1 A/AA checks (axe-core) on 13 pages

Page	Violations	Rules (impact, elements)	Rules passed
Home	0	-	21
Vote sign-in	0	-	24
Check receipt	0	-	23
Demo guide	0	-	23
Privacy	0	-	20
Contact	1	link-in-text-block (serious, 1)	24
Admin sign-in	0	-	21
Elections	0	-	21
Voters	0	-	21
Results	0	-	21
DB Studio	0	-	21
Audit ledger	0	-	21
Mail outbox	0	-	21

Automated checks find only part of the accessibility problems; they are evidence, not certification.

Table. Page load on the local server (median of 5 loads, ms)

Page	First contentful paint	DOM loaded	Load	JS transferred KB
Home	128	173.1	173.6	280.6
Vote sign-in	140	182.9	183.2	313.6
Check receipt	140	172.6	173	284.8
Demo guide	132	166.6	167.2	288.1
Privacy	132	165.7	165.9	285.2
Contact	136	172.4	172.8	284.6
Admin sign-in	136	170.9	171.6	283.4

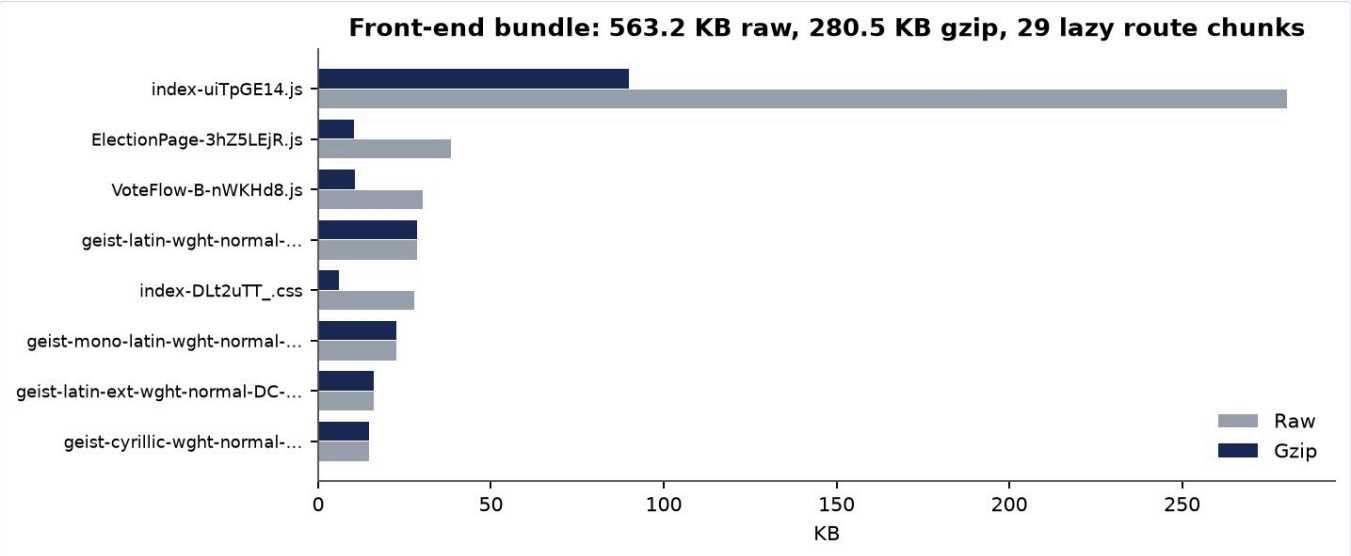


Figure 18. Front-end bundle. Largest files in the production build. Routes are split into chunks that load on demand.

Table 14. Live deployment

Measure	Value
Live site	https://polyvote-bacse202.onrender.com
Website checklist	None pass, None N/A, None fail
Health check median / min / max (ms)	112 / 102 / 2244

Measure	Value
First request (cold start possible) ms	22817
Header content-security-policy	absent
Header x-content-type-options	absent
Header x-frame-options	absent
Header referrer-policy	absent
Header permissions-policy	absent
Header strict-transport-security	absent
Header cache-control	absent

3.15 Code

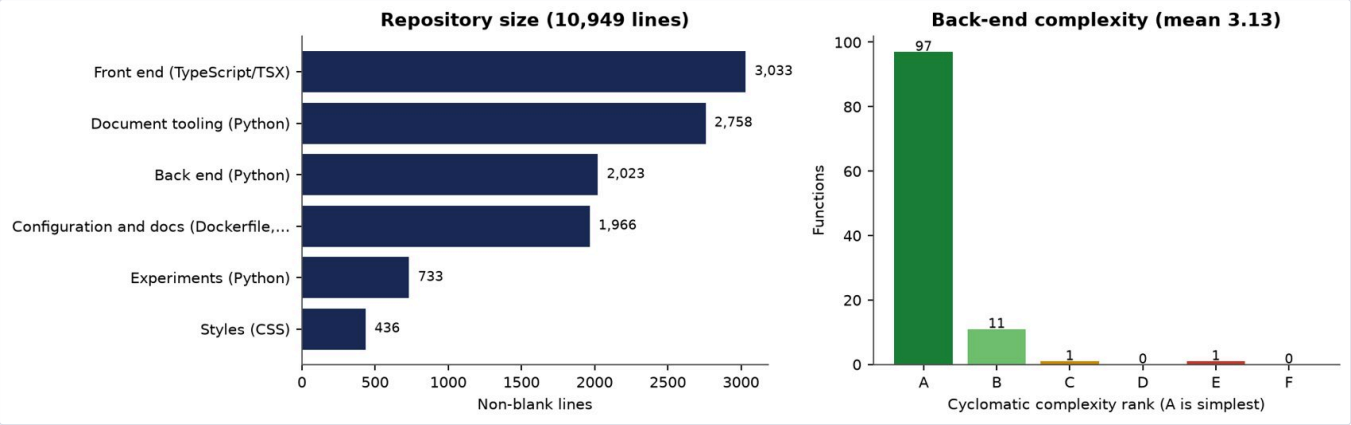


Figure 19. Code size and complexity. Left: lines of code by component. Right: distribution of cyclomatic complexity ranks across back-end functions (radon).

Table 15. Code, structure and static analysis

Measure	Value
Lines (non-blank), total	10,949
Back end (Python)	2,023
Front end (TypeScript/TSX)	3,033
Styles (CSS)	436
Experiments (Python)	733
Document tooling (Python)	2,758
Configuration and docs (Dockerfile, YAML, Markdown)	1,966
API routes	36
Election tables / triggers / views	5 / 1 / 2
CHECK / UNIQUE+PK / FOREIGN KEY	2 / 6 / 5
Front-end pages	23
Python dependencies	5
npm dependencies	7
Mean cyclomatic complexity (back end)	3.13
Bandit findings (severity)	6 {'MEDIUM': 3, 'LOW': 3}
npm audit	{'info': 0, 'low': 0, 'moderate': 0, 'high': 0, 'critical': 0, 'total': 0}
pip-audit (hosted requirements)	0 advisories in 20 packages
pip-audit (with the legacy Streamlit UI)	39 advisories, all in pillow (fixed in 13.0.0)

4. Comparison with related systems

4.1 What was measured and what was read

We read **33 systems** (hosted services, open-source projects, research systems and independent security analyses) from their own pages, READMEs and papers; every cell of the feature matrix has a source URL in `docs/review3/uniqueness_evidence.md`. We **did not run** those systems. What we could measure is the *integrity mechanism* each documents, by running the same attacks against the design class it belongs to (Section 2.3). The first column of Table 20 states the documented mechanism and the source of the classification.

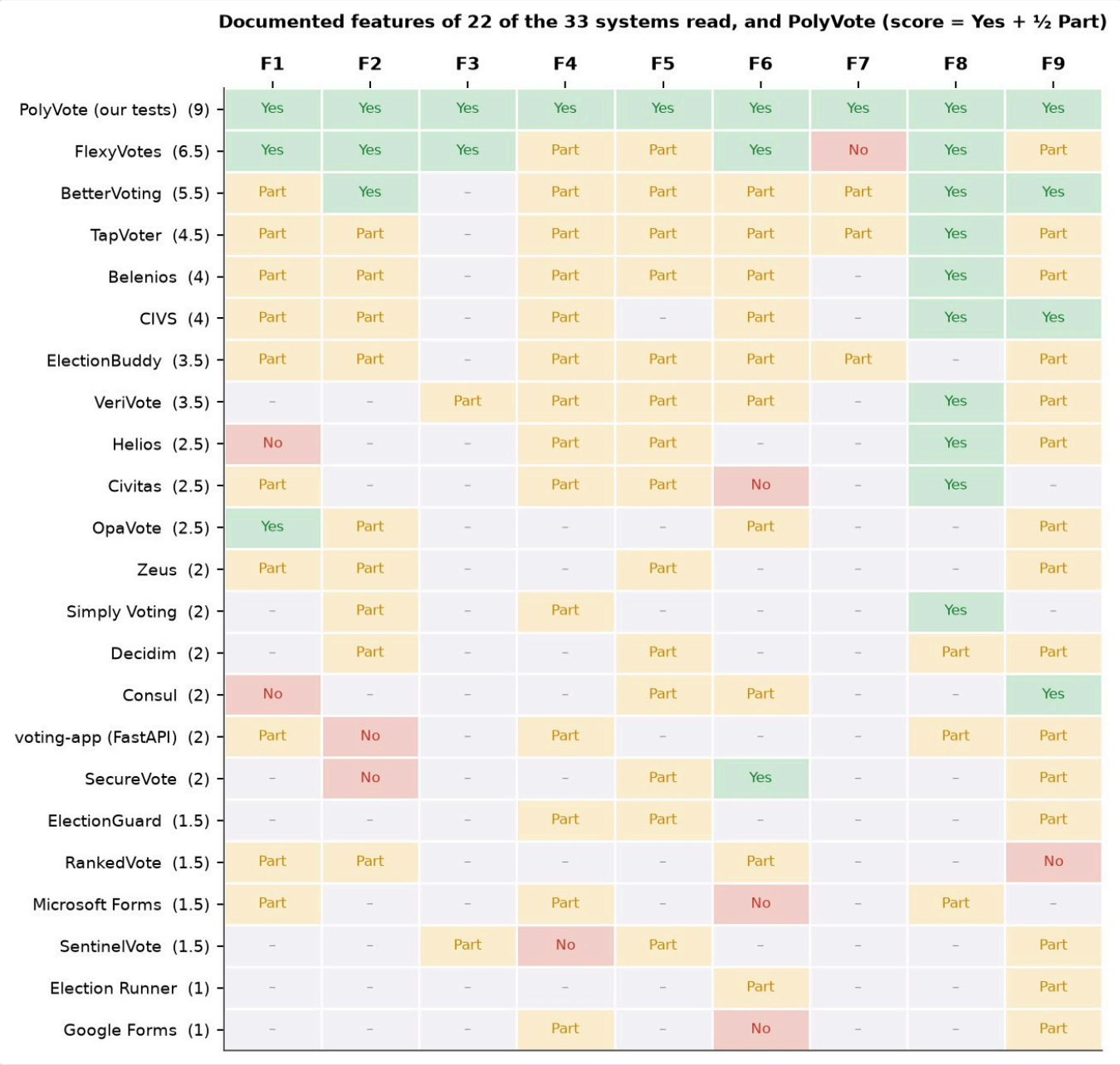


Figure 20. Documented feature coverage of related systems. From each system's own page, README or paper (URL for every cell in the evidence file). F1 ranked counting, F2 many organisations, F3 database rules, F4 unlinked ballots, F5 keyed ledger and sweep, F6 email OTP, F7 QR voting, F8 receipt, F9 easy to run. – means not documented, not absent. PolyVote's row rests on the tests in this dossier.

Table 16. Documented features of the 33 systems read (F1-F9), from their own pages

System	Type	F1	F2	F3	F4	F5	F6	F7	F8	F9
Helios Voting (helios-server)	open-source, verifiable (E2E)	no	not documented	not documented	partial	partial	not documented	not documented	yes	partial
Belenios	open-source, verifiable (E2E), hosted platform + self-host	partial	partial	not documented	partial	partial	partial	not documented	yes	partial
Civitas	research prototype (coercion-resistant, verifiable)	partial	not documented	not documented	partial	partial	no	not documented	yes	not documented
ElectionGuard (Microsoft / Election-Tech-Initiative)	open-source SDK for E2E verifiable elections	not documented	not documented	not documented	partial	partial	not documented	not documented	not documented	partial

System	Type	F1	F2	F3	F4	F5	F6	F7	F8	F9
Zeus (GRNET)	open-source, verifiable, hosted service (Greek research network)	partial	partial	not documented	not documented	partial	not documented	not documented	not documented	partial
Verificatum (vmn mix-net libraries)	open-source cryptographic libraries (mix-net)	not documented	not documented	not documented	not documented	not documented	not documented	not documented	not documented	not documented
Vocdoni (vocdoni-node)	open-source blockchain (Vochain) voting protocol	not documented	not documented	not documented	not documented	not documented	not documented	not documented	not documented	partial
Aragon (aragon/client, Aragon 1)	open-source DAO / blockchain governance app	not documented	not documented	not documented	not documented	not documented	not documented	not documented	not documented	not documented
CIVS (Condorcet Internet Voting Service, Cornell)	free hosted service + open-source (MIT) software	partial	partial	not documented	partial	not documented	partial	not documented	yes	yes
OpaVote	commercial/freemium SaaS	yes	partial	not documented	not documented	not documented	partial	not documented	not documented	partial
ElectionBuddy	commercial SaaS	partial	partial	not documented	partial	partial	partial	partial	not documented	partial
Simply Voting	commercial SaaS	not documented	partial	not documented	partial	not documented	not documented	not documented	yes	not documented
Election Runner	commercial SaaS	not documented	not documented	not documented	not documented	not documented	partial	not documented	not documented	partial
TapVoter	commercial/freemium SaaS aimed at student councils	partial	partial	not documented	partial	partial	partial	partial	yes	partial
RankedVote	commercial SaaS (ranked-choice only)	partial	partial	not documented	not documented	not documented	partial	not documented	not documented	no
BetterVoting (Equal-Vote/bettervoting, formerly star-server)	open-source (AGPL-3.0) + free hosted service bettervoting.com	partial	yes	not documented	partial	partial	partial	partial	yes	yes
Google Forms	commercial free general-purpose forms	not documented	not documented	not documented	partial	not documented	no	not documented	not documented	partial
Microsoft Forms	commercial general-purpose forms (part of Microsoft 365)	partial	not documented	not documented	partial	not documented	no	not documented	partial	not documented
Voatz (mobile voting app) - USENIX Security 2020 analysis	commercial proprietary mobile voting app (independent analysis)	not documented	not documented	not documented	not documented	partial	not documented	not documented	not documented	no
Democracy Live OmniBallot - USENIX Security 2021 analysis	commercial proprietary online ballot delivery/return (independent analysis)	not documented	not documented	not documented	no	not documented	not documented	not documented	not documented	no
Decidim (decidim-elections + Bulletin Board)	open-source civic participation platform (AGPL-3.0)	not documented	partial	not documented	not documented	partial	not documented	not documented	partial	partial
Consul Democracy	open-source civic participation platform (AGPL-3.0)	no	not documented	not documented	not documented	partial	partial	not documented	not documented	yes
deOpty/flexyvotes (Django election + paid-voting platform)	student/hobby-scale open repo (single developer, created 2026-08-10) - NEAREST MATCH FOUND	yes	yes	yes	partial	partial	yes	no	yes	partial
nnnnnndev/voting-app (FastAPI, faculty-meeting voting)	student/hobby repo (departmental tool)	partial	no	not documented	partial	not documented	not documented	not documented	partial	partial
9022574361/Online_Voting_System ('VeriVote')	student/hobby repo	not documented	not documented	partial	partial	partial	partial	not documented	yes	partial
DikshantMaskare/securevote-online-voting-system	student/hobby repo	not documented	no	not documented	not documented	partial	yes	not documented	not documented	partial
RuchikaaVerma/SentinelVote-AI	student/hobby repo	not documented	not documented	partial	no	partial	not documented	not documented	not documented	partial
d-kanishka/Secure_Voting_System	student/hobby repo	no	not documented	no	no	not documented	no	no	not documented	partial
Prakhar-Pandey20/Online-Voting-System-otp-version	student/hobby repo	not documented	not documented	partial	no	not documented	partial	not documented	not documented	not documented
jevk/RCVElectionSystem	student/hobby repo (gaming community)	partial	not documented	not documented	not documented	not documented	not documented	not documented	not documented	partial

System	Type	F1	F2	F3	F4	F5	F6	F7	F8	F9
dryairship/online-election-manager	student project (IIT Kanpur course ESC101A)	not documented	not documented	not documented	not documented	not documented	partial	not documented	not documented	partial
Dharanibalan45/Online-Voting-Platform	student/hobby repo	not documented	not documented	not documented	not documented	not documented	partial	not documented	not documented	not documented
Lervez5/Kuraone	student/hobby-scale repo (proprietary notice)	not documented	not documented	not documented	not documented	partial	not documented	not documented	not documented	no
PolyVote (this project)	student project, open on GitHub	yes	yes	yes	yes	yes	yes	yes	yes	yes

4.2 Documented integrity mechanisms mapped to measured design classes

System	Documented mechanism	Closest measured class	Measured detection of the class	What this does not capture
FlexyVotes (Django, PostgreSQL)	Per-organisation SHA-256 hash-chained audit log in the same PostgreSQL database; append-only triggers on ballots and audit rows that refuse changes even from a raw SQL session; Merkle bulletin board and signed results; hashed single-use email OTP.	P2 plain chain, plus the append-only trigger class	6/31 (chain) and 8/31 prevented (triggers, limited writer)	The Merkle bulletin board and signed results are not emulated; they let verifiers detect ballot edits independently of the server.
VeriVote (Node, React, MongoDB)	SHA-256 chain in MongoDB with an audit endpoint and a manual tamper demo.	P2 plain chain	6/31	Not emulated: its OTP and receipt hash.
SecureVote, SentinelVote, Kuraone (student repositories)	A hash-linked ledger of votes.	P2 plain chain	6/31	Single instance; documentation only.
BetterVoting	Audit log of administrator actions (sharing a link, revealing a ballot link); ballots stored with identity removed.	P0 counters (no integrity check of stored ballots is documented)	11/31 (counters by eye)	Its audit log records actions taken through the application, which database-level tampering bypasses.
Google Forms, Microsoft Forms, typical CRUD apps	Responses in a table; no tamper evidence documented.	P0 CRUD, officer compares counters	11/31	
OpaVote, ElectionBuddy, Simply Voting, Election Runner, TapVoter	Vendor-run services with voter audit reports, raw-ballot downloads and receipt codes.	Not emulated	-	The vendor holds the data; the integrity mechanisms are internal and not documented in enough detail to emulate.
Helios, Belenios, ElectionGuard, Civitas, Zeus	End-to-end verifiable: ballots are encrypted and posted to a public bulletin board; voters and auditors can verify inclusion and the tally.	Different architecture	-	A stored-ballot edit is detected by any voter who checks and by public audit; PolyVote detects it only through its own sweep. These systems are stronger on proving what a ballot says.
PolyVote final	Keyed hash chain in a separate store, six sweep rules, database rules, keyed roster seal.	P7	**28/31 (28/28 in scope)**	Cannot prove what a ballot says to the voter (not end-to-end verifiable).

Reading this table fairly. The classes are emulations of documented mechanisms, not the products. They show how far each *mechanism* goes against storage-level tampering. They say nothing about features outside that scope, and the products may have protections that are not documented.

4.3 Where PolyVote stands

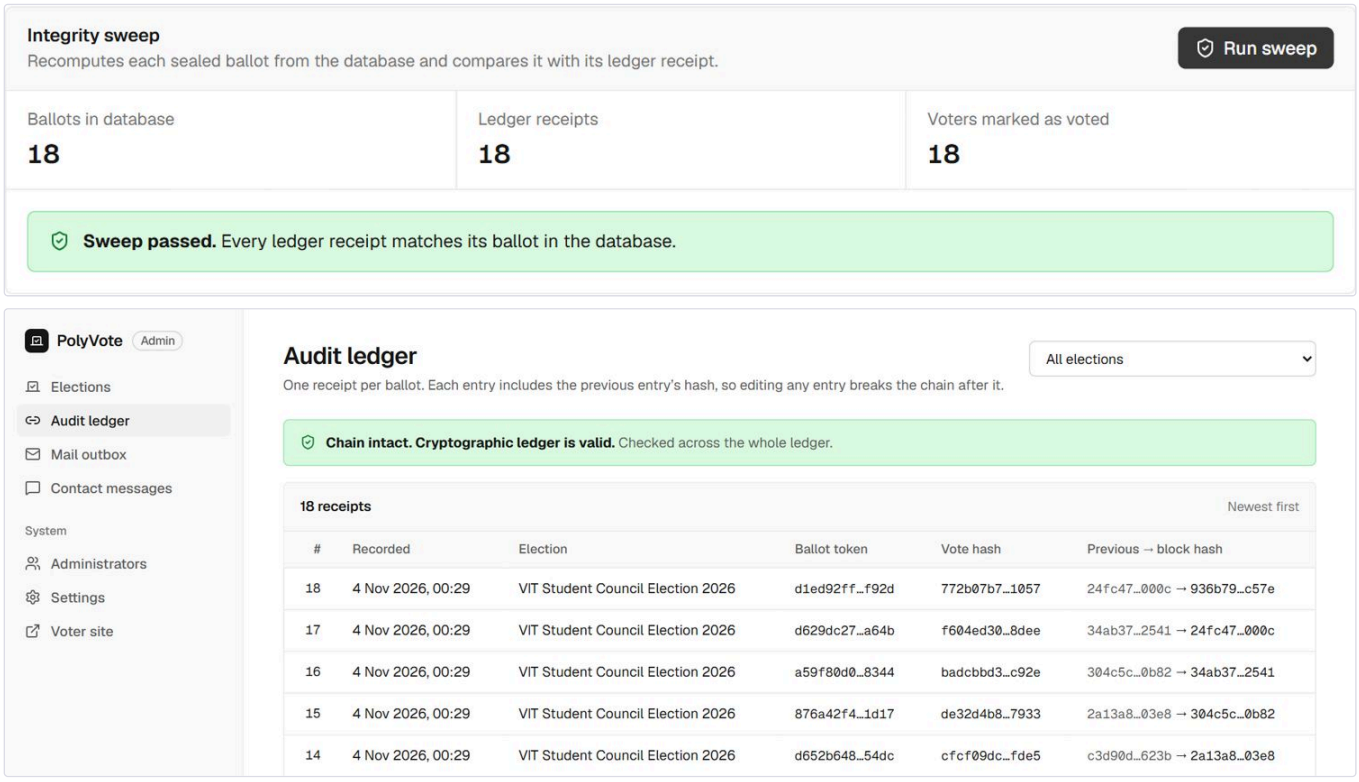
- **Combination of controls.** The documented systems each have part of what PolyVote combines: ranked counting and organisations (OpaVote, FlexyVotes, BetterVoting), database rules and append-only triggers (FlexyVotes), a hash-chained ledger (FlexyVotes, VeriVote and others), emailed one-time codes (FlexyVotes, SecureVote), QR sharing (TapVoter, BetterVoting). Across the 33 systems we read, none documents all of them together; PolyVote adds a keyed chain in a separate store, a reconciling sweep with a keyed roster seal, QR scan-to-vote, and published executable attack results.
- **Measured coverage.** Against the same 31 attacks, the design classes of the closest projects detect 6 (plain chain) and prevent 8 (append-only triggers) while PolyVote detects 28 (28 in scope).
- **Cost.** Free, open source, one command to run, runs on a laptop or a free hosting plan; the whole back end is about 2,023 lines.

4.4 Where related systems are stronger

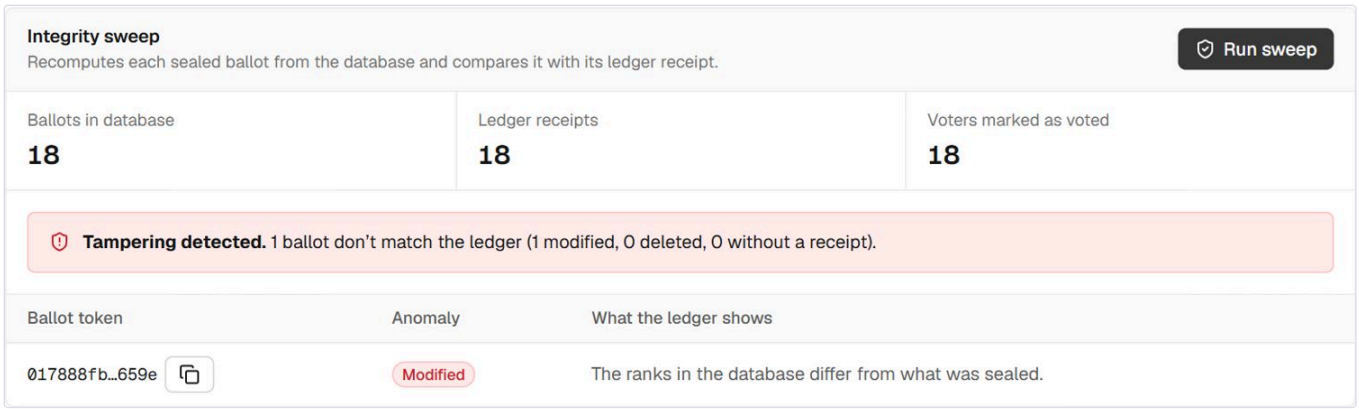
- **Verifiability of ballot content.** Helios, Belenios, ElectionGuard and Civitas let a voter confirm that the ballot recorded is the one they cast and let anyone audit the tally; PolyVote's receipt proves only that a ballot is recorded and counted.
- **Untrusted administrator.** Cryptographic systems keep working when the administrator is hostile; PolyVote assumes the key holder is honest.
- **Maturity.** Helios, Belenios and CIVS have years of use and public analysis; PolyVote has had none.
- **Append-only enforcement and a public bulletin board (FlexyVotes)** are stronger prevention and public-verification features than PolyVote's file or Atlas ledger.

5. The application under attack: screenshots

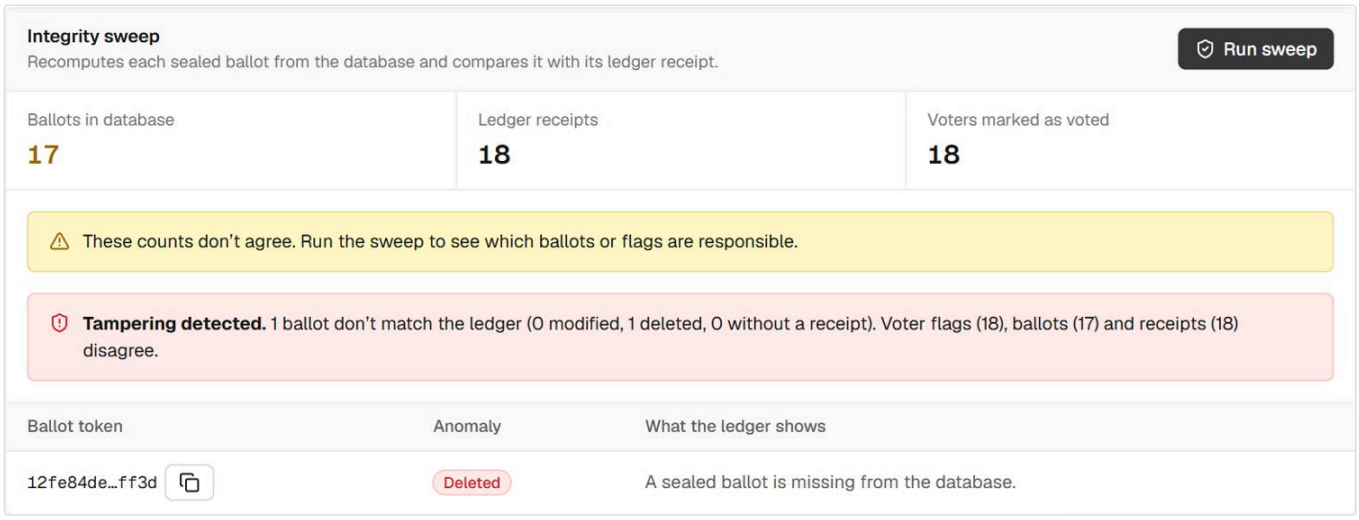
Each attack below was applied to the files of a running copy of the app (keyed ledger, demo election with 30 voters and 18 ballots) and the sweep was run in the browser. The monospaced line under each image is the page text.



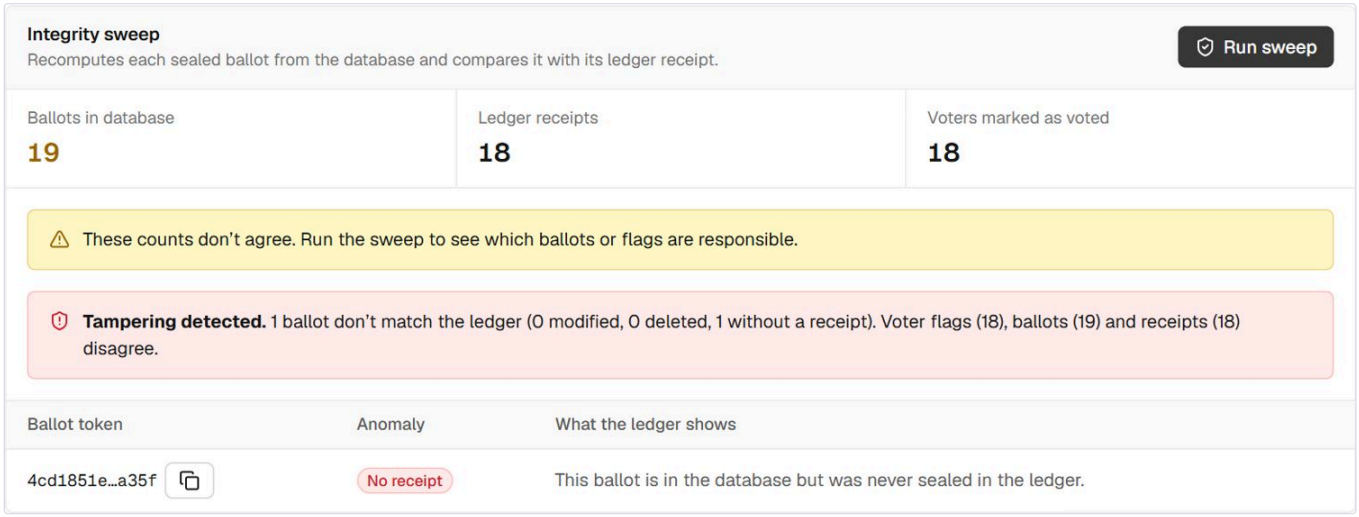
Screenshot 1. Clean election: sweep passes and the chain is intact.



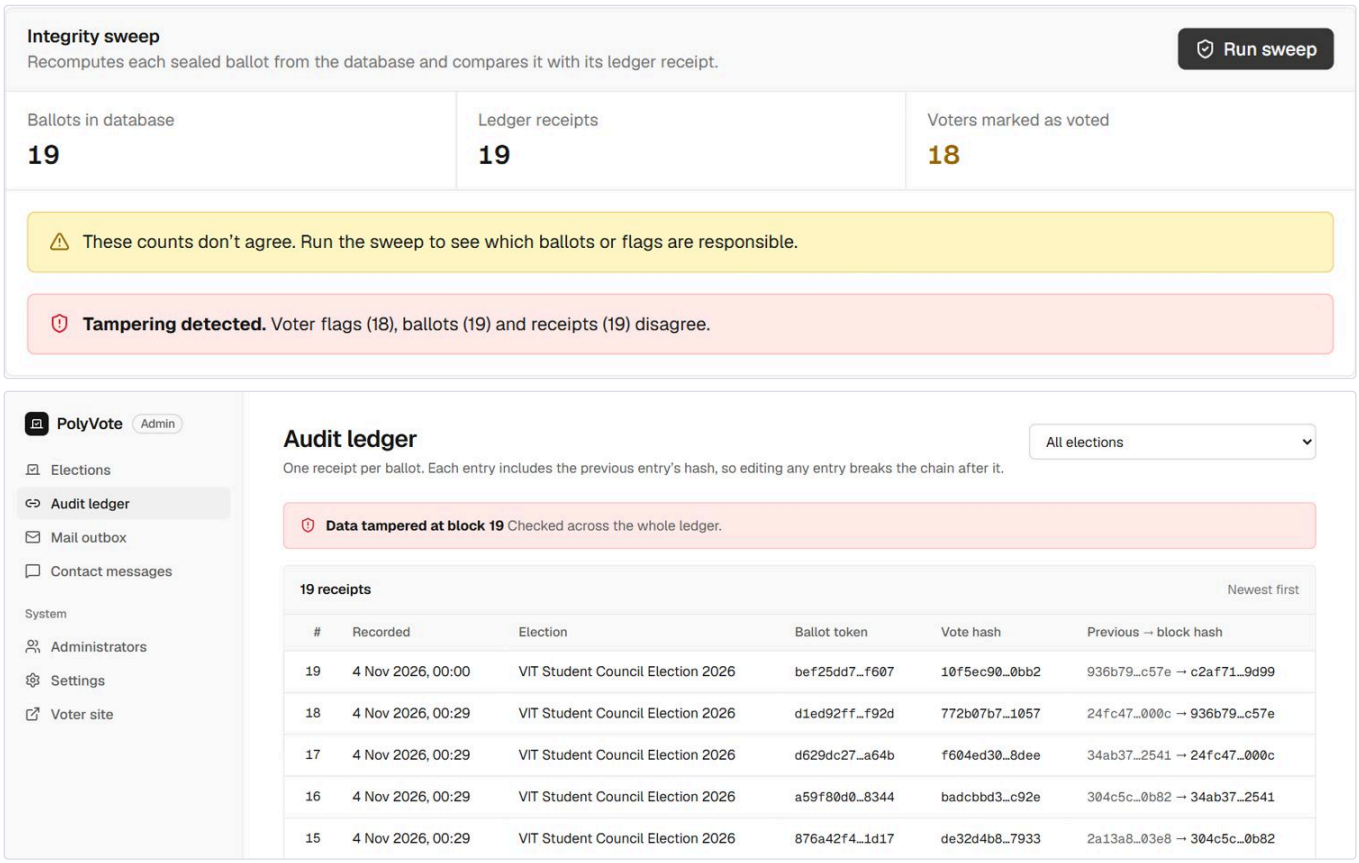
Screenshot 2. C1 Swap the 1st and 2nd choice on one ballot. Rule 1 (modified) names the ballot.



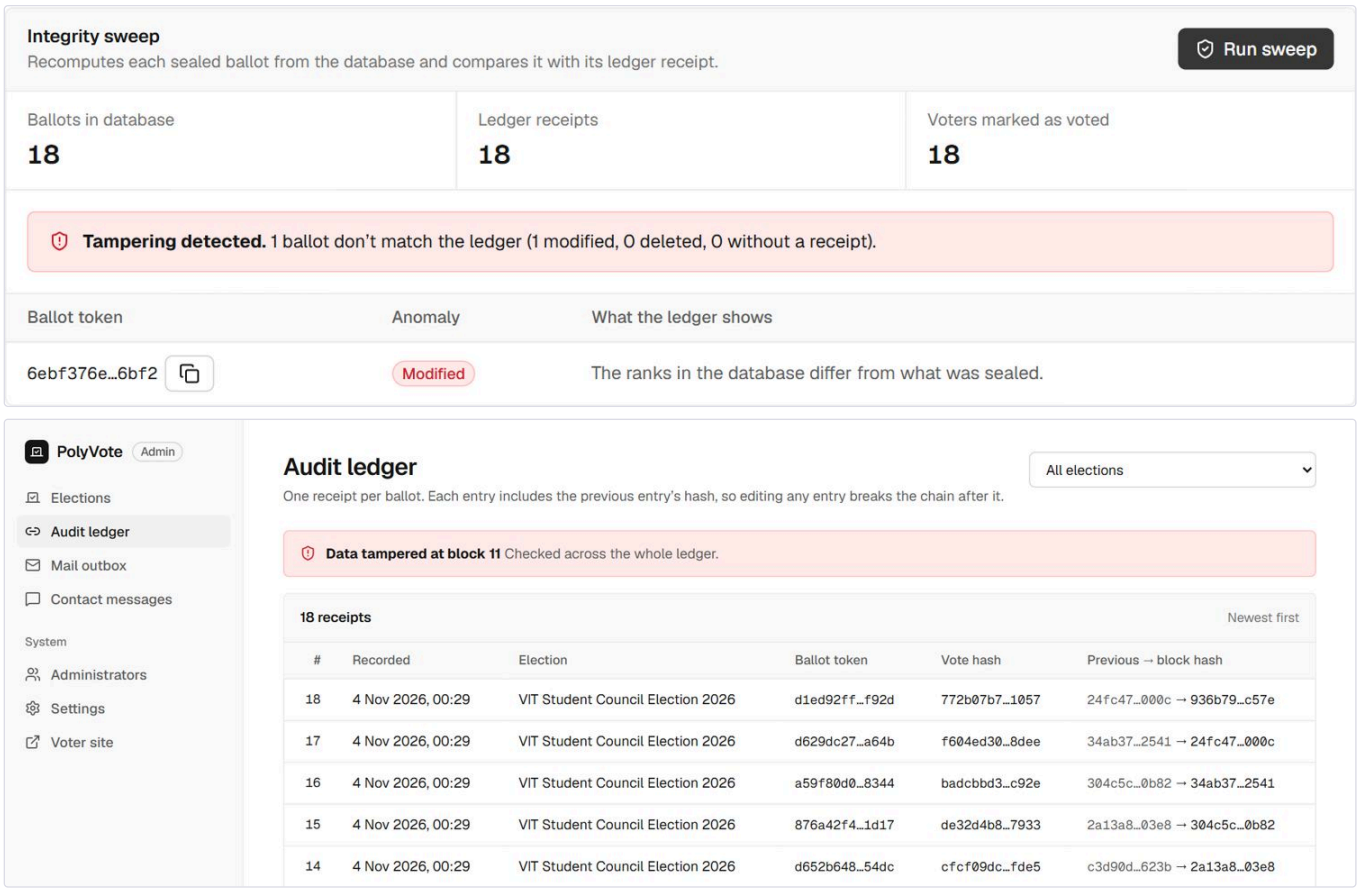
Screenshot 3. C3 Delete one ballot. Rule 2 (deleted) and the count check fire.



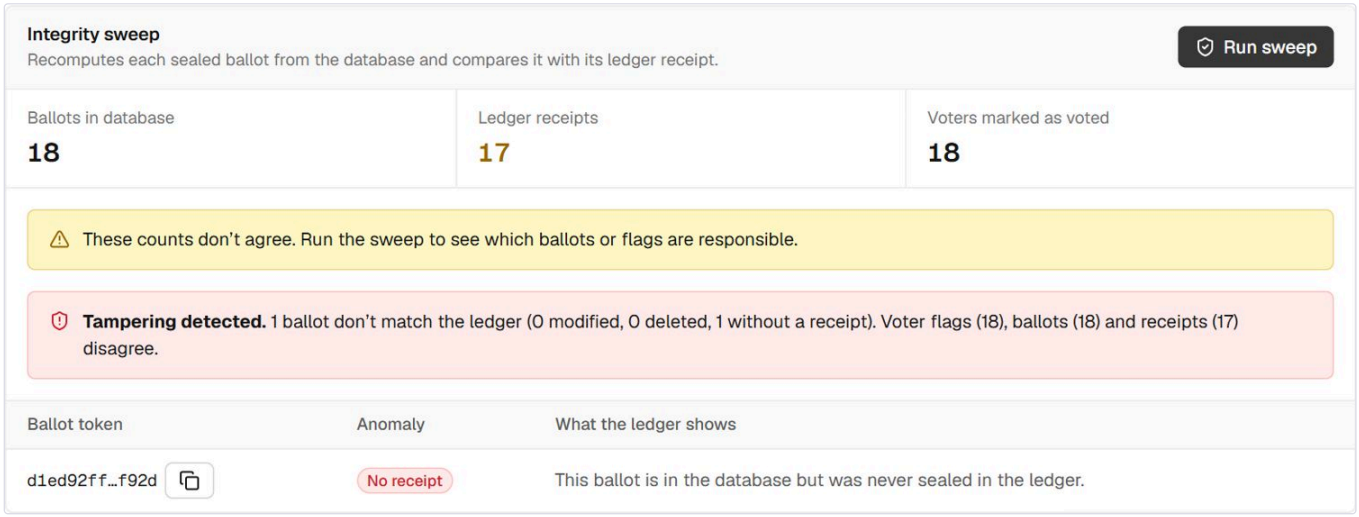
Screenshot 4. C5 Inject a ballot with no receipt. Rule 3 (injected) and the count check fire.



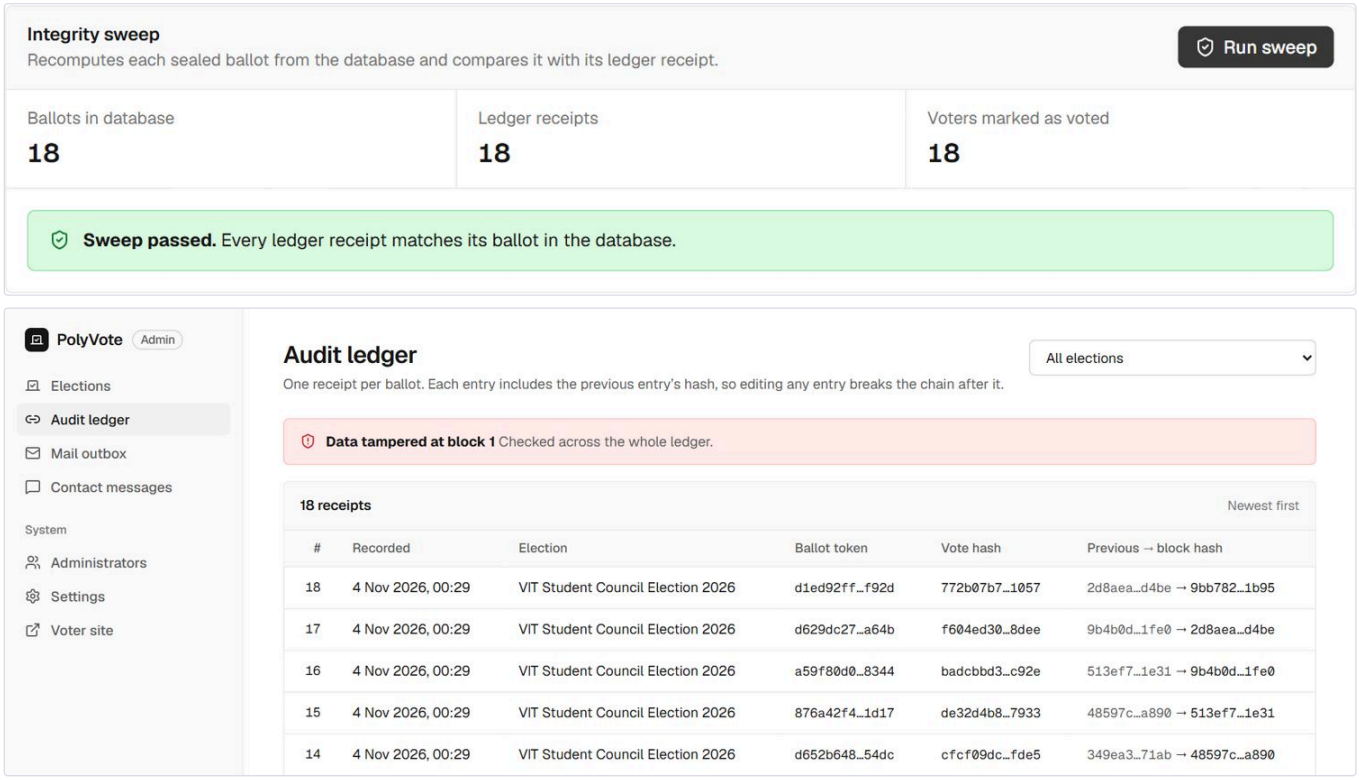
Screenshot 5. C6 Ballot stuffing with a forged plain-hash receipt. The forged receipt is not keyed, so the chain check fails at the forged block.



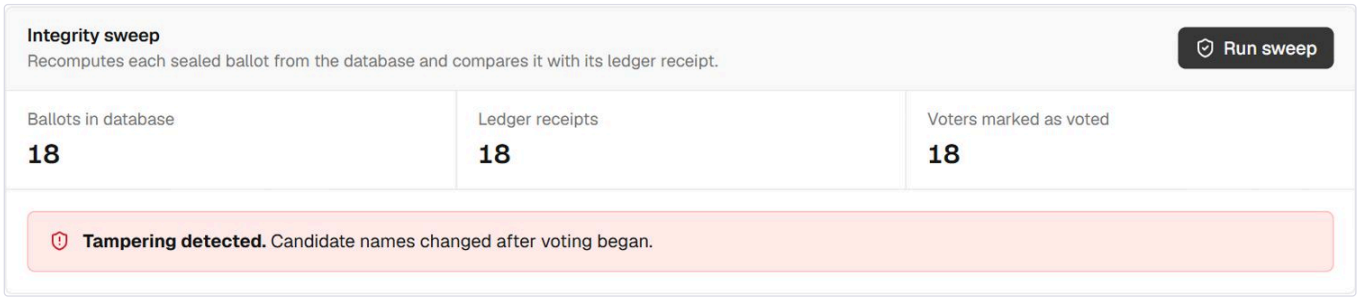
Screenshot 6. C7 Edit a ledger block. The chain check names the edited block.



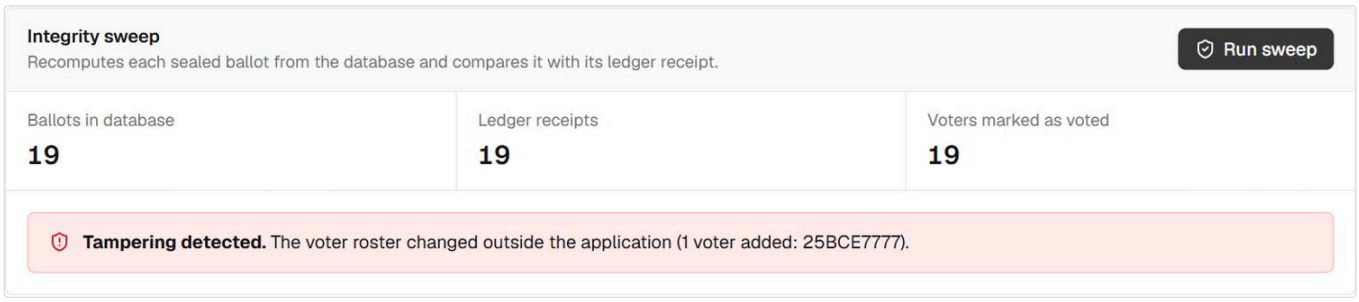
Screenshot 7. C9 Truncate the last ledger block. Truncation leaves ballots without receipts; the sweep reports them.



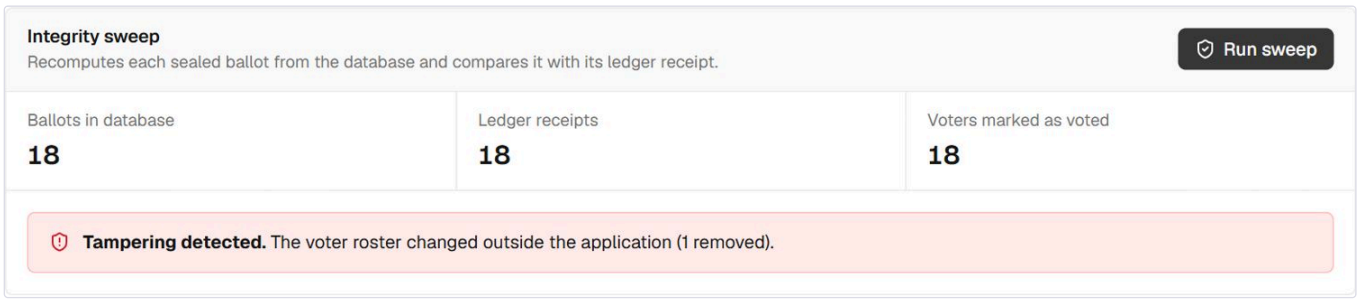
Screenshot 8. C10 Rebuild the whole chain without the key. A chain rebuilt without the key fails the keyed check.



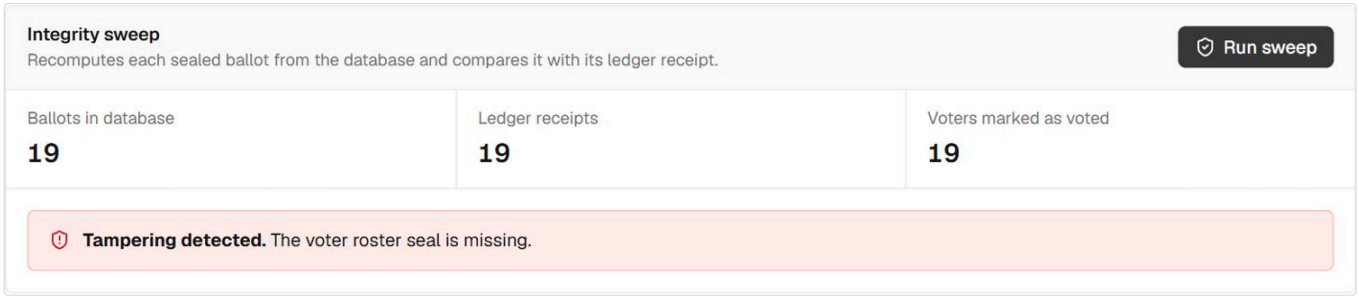
Screenshot 9. C11 Swap two candidates' names after voting. Rule 4: candidate names no longer match the sealed roster.



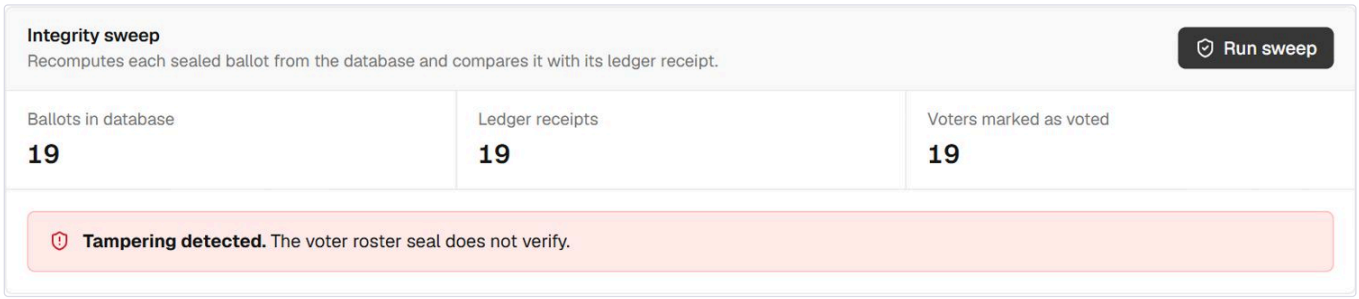
Screenshot 10. C13 Ghost voter added behind the application and used to vote. Rule 6 (new): a voter added behind the application is named.



Screenshot 11. C14 Delete an unvoted voter from the roster. Rule 6: a removed voter is counted.



Screenshot 12. C16 Delete the roster seal, then add a ghost voter. Deleting the roster seal is itself detected.



Screenshot 13. C17 Forge the roster seal without the key. A seal forged without the key does not verify.

Integrity sweep

Recomputes each sealed ballot from the database and compares it with its ledger receipt.

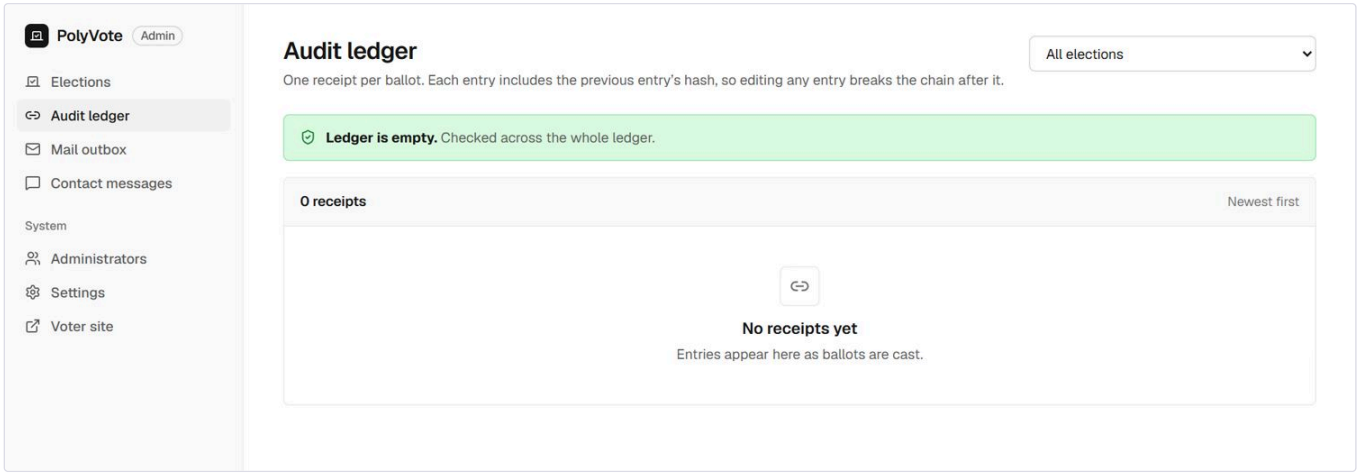
Run sweep

Ballots in database	Ledger receipts	Voters marked as voted
18	0	18

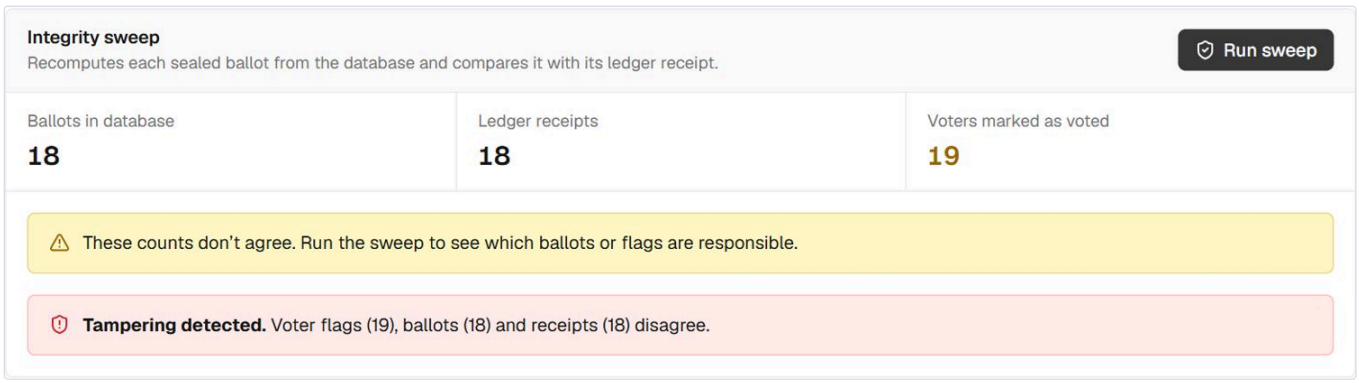
⚠ These counts don't agree. Run the sweep to see which ballots or flags are responsible.

Tampering detected. 18 ballots don't match the ledger (0 modified, 0 deleted, 18 without a receipt). Voter flags (18), ballots (18) and receipts (0) disagree.

Ballot token	Anomaly	What the ledger shows
017888fb…659e	No receipt	This ballot is in the database but was never sealed in the ledger.
0b6187ba…508b	No receipt	This ballot is in the database but was never sealed in the ledger.
12fe84de…ff3d	No receipt	This ballot is in the database but was never sealed in the ledger.
22d93558…b98a	No receipt	This ballot is in the database but was never sealed in the ledger.
41e4c0cc…9386	No receipt	This ballot is in the database but was never sealed in the ledger.
67cb13aa…faf1	No receipt	This ballot is in the database but was never sealed in the ledger.
6ebf376e…6bf2	No receipt	This ballot is in the database but was never sealed in the ledger.
7843ecb2…d178	No receipt	This ballot is in the database but was never sealed in the ledger.
78d2a0c5…cab3	No receipt	This ballot is in the database but was never sealed in the ledger.
876a42f4…1d17	No receipt	This ballot is in the database but was never sealed in the ledger.
a59f80d0…8344	No receipt	This ballot is in the database but was never sealed in the ledger.
b2e64597…d259	No receipt	This ballot is in the database but was never sealed in the ledger.
d1ed92ff…f92d	No receipt	This ballot is in the database but was never sealed in the ledger.
d629dc27…a64b	No receipt	This ballot is in the database but was never sealed in the ledger.
d652b648…54dc	No receipt	This ballot is in the database but was never sealed in the ledger.
dc937e2a…0340	No receipt	This ballot is in the database but was never sealed in the ledger.
eae32d7a…8f9a	No receipt	This ballot is in the database but was never sealed in the ledger.
f8ffb880…6990	No receipt	This ballot is in the database but was never sealed in the ledger.

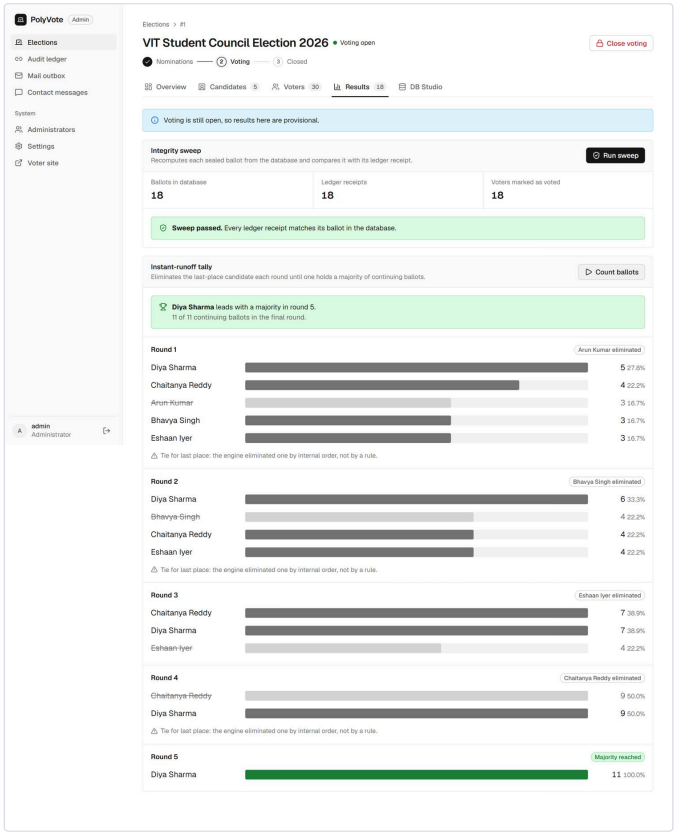


Screenshot 14. C18 Wipe the whole ledger. Wiping the ledger leaves every ballot without a receipt.

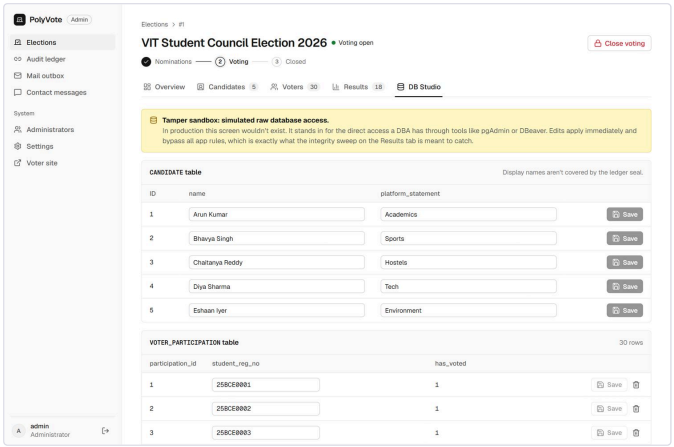


Screenshot 15. C26 Mark an unvoted voter as voted with no ballot. The count check fires: a flag without a ballot.

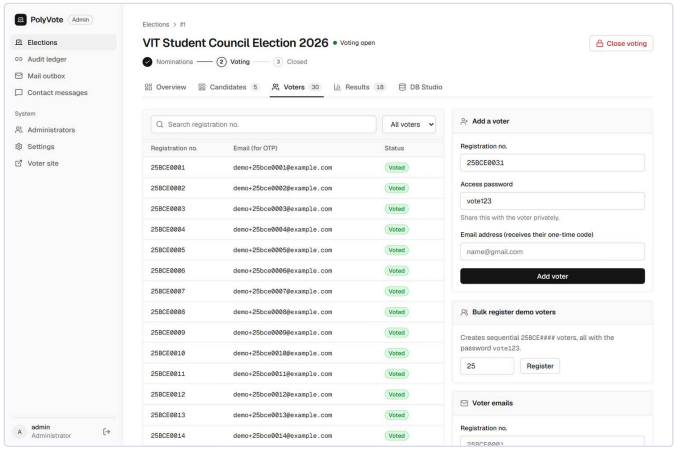
Other views



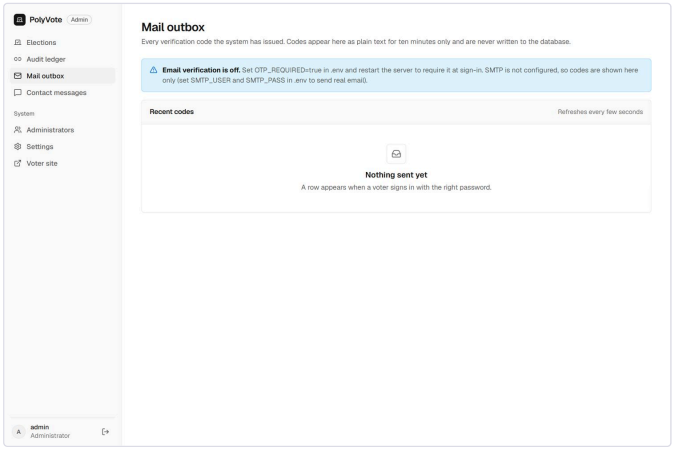
Screenshot 16. V_results.



Screenshot 17. V_studio.



Screenshot 18. V_voters.



Screenshot 19. V_outbox.

6. Threats to validity and limitations

- **We wrote the attacks.** The scenarios are ours, so the suite is biased towards what we thought of. We widened it from 13 to 31 and describe each in Table 1; independent review is needed.
- **Design classes are emulations.** Related systems were not run. The emulation approach was validated against our own real Review 2 code (13 of 13 agreement) but not against third-party products.
- **Out-of-model attackers.** An insider with the ledger key (C28, C29) is not detected; protecting against it needs an external witness for the chain head, a managed secret store, or both.
- **Unsealed text.** Candidate platform statements are not covered by the sealed roster hash (C23).
- **Voter privacy limits.** An attacker who copies the database before and after a single cast can link that voter to that ballot (Figure 17); the administrator's simulation tool casts in roster order (demo only).
- **Single machine, synthetic data.** Performance numbers come from one laptop; ledger measurements use the file backend, which rewrites the file on each append. The Atlas figures come from an earlier run on a mobile hotspot.
- **No user study yet.** Usability and delivery-time claims await the pilot election with real voters.
- **Known engineering gaps.** No lock-out on repeated wrong passwords, no session expiry, user enumeration at sign-in, no security headers on the API, SQLite foreign keys per connection (A5), single node.

7. Reproducibility

Repository	https://github.com/Tharakeshtj/polyvote-election-platform
Attack matrix and profiles	`experiments/full_eval/engine.py`
Security, privacy, accuracy, tally, passwords, scaling	`experiments/full_eval/extras.py`
Static analysis, bundle, live site	`experiments/full_eval/static_metrics.py`
Screenshots, accessibility, page load	`experiments/full_eval/shots.py`
Figures and tables	`experiments/full_eval/figures.py`
Core suite (prevention, API, one-time code)	`experiments/run_experiments.py`
Website checklist	`experiments/web_checklist.py` (live: 18 pass, 3 not applicable, 0 fail)
Related-systems evidence	`docs/review3/uniqueness_evidence.md` and `.json` (a source URL for every cell)

Appendix A. Full tables

Table 17. Detectors that fire for each attack (keyed chain)

ID	Chain check	Rule 1 modified	Rule 2 deleted	Rule 3 injected	Rule 4 candidate seal	Rule 5 count mismatch	Rule 6 voter seal	Eyeballing counters	Any automatic detector, plain chain
C1		•							•
C2		•							•
C3			•			•		•	•
C4			•			•		•	•
C5				•		•		•	•
C6	•								
C7	•	•							•
C8	•			•		•		•	•
C9				•		•		•	•
C10	•								
C11					•				•
C12						•		•	•
C13							•		•
C14							•		•
C15							•		•
C16							•		•
C17							•		
C18				•		•		•	•
C19	•							•	•
C20	•								•
C21	•								•
C22	•								•
C23									
C24					•				•
C25					•				•
C26						•		•	•
C27	•								
C28									
C29									
C30				•		•		•	•
C31			•			•		•	•

Table 18. Emulating the Review 2 design from the matrix reproduces the real Review 2 code (13/13)

Scenario	Real Review 2 code (measured earlier)	Emulated from the detector matrix	Agree
C1	detected	detected	yes
C2	detected	detected	yes
C3	detected	detected	yes
C4	detected	detected	yes
C5	detected_manual_only	detected_manual_only	yes

Scenario	Real Review 2 code (measured earlier)	Emulated from the detector matrix	Agree
C6	undetected	undetected	yes
C7	detected	detected	yes
C8	detected	detected	yes
C9	detected_manual_only	detected_manual_only	yes
C10	undetected	undetected	yes
C11	undetected	undetected	yes
C12	detected_manual_only	detected_manual_only	yes
C13	undetected	undetected	yes